

Вопросы к экзамену по дисциплине “Технология проектирования приложений”

1 - Методы получения описаний предметной области.	3
2 - Выделение важнейших компонентов текста.	5
3 - Верификация текста.	6
4 - Текстовое описание сущностей, атрибутов, значений атрибутов.	7
5 - Текстовое описание процессов, участников и действий.	9
6 - ER-диаграмма. Сущности и отношения.	10
7 - ER-диаграмма. Типы связей сущностей.	11
8 - ER-диаграмма. Атрибуты сущностей. Сильные и слабые сущности.	12
9 - ER-диаграмма. Ключевые атрибуты. Простые и составные ключевые атрибуты.	14
10 - ER-диаграмма. Бинарные, тернарные и n-арные связи.	16
11 - ER-диаграмма. Проблема “единичного значения атрибута”. Пропущенные сущности, отношения и атрибуты.	17
12 - ER-диаграмма. Составление описания по тексту. Проверка согласования диаграммы и исходного текстового описания.	18
13 - Цели описания ситуативного взаимодействия.	20
14 - Изолированность ситуативного взаимодействия.	22
15 - Сценарии использования (use case). Перечень разделов описания сценариев использования.	23
16 - Позитивные и негативные исходы use case.	24
17 - Основной и альтернативный сценарий use case.	26
18 - Диаграммы сценариев использования. Синтаксис графического представления. Действия и ассоциации.	28
19 - Ассоциации, обобщения, включения и расширения.	30
20 - Акторы. Отображение обобщения акторов.	34
21 - Include и extend. Способы использования и ключевые операции.	35
22 - Плоская и иерархическая нотация в представлении use case.	37
23 - Понятие бизнес-процесса.	39
24 - Основные и вспомогательные бизнес-процессы.	41
25 - Повторяемость бизнес-процессов.	42
26 - Участники бизнес-процессов.	44
27 - Определение полноты и достаточности описания бизнес-процесса.	46
28 - Позитивные и негативные исходы бизнес-процессов.	48
29 - Условия и ветвления в бизнес-процессах. Не бинарные ветвления.	50
30 - Виды BPM-нотаций. BPMN, IDEF0, блок-схемы.	53
31 - Синтаксис BPM 2.0. Дорожки, события.	55
32 - Синтаксис BPM 2.0. Действия, шлюзы.	57
33 - Синтаксис BPM 2.0. Таймеры в бизнес-процессах.	59
34 - Синхронизация бизнес-процессов.	60
35 - Программная реализация бизнес-процессов.	61
36 - Реестр бизнес-процессов. Цели формирования реестра.	62

37 - Запуск бизнес-процессов от пользователя. Реализация шагов бизнес-процессов.	63
38 - Проектирование программной реализации бизнес-процессов. Графическое представление.	65
39 - Уведомления в бизнес-процессах. Правила формирования уведомлений.	67
40 - Статусы состояний бизнес-процессов. Цели и задачи формирования статусов.	69
41 - Проектирование экранных форм шагов бизнес-процесса. Создание эскизов форм.	70
42 - Описание данных бизнес-процесса. Формирование общего описания данных.	71
43 - Реализация описания данных. Пример.	72
44 - Формат JSON. JSON-SCHEMA.	73
45 - Формат JSON. Синтаксис.	76
46 - Формат JSON. Массивы данных.	78

1 - Методы получения описаний предметной области.

Описание предметной области — процесс сбора, анализа и систематизации информации о конкретной сфере деятельности, проблеме или системе.

Один из первых этапов проектирования информационных систем, разработки ПО, научных исследований и управления бизнес-процессами.

Методы получения описания предметной области:

Анализ литературы.

Изучение письменных источников (научных, технических, нормативных), связанных с предметной областью.

Подходы:

1. Систематический обзор — анализ множества публикаций по теме.
2. Критический разбор — оценка достоверности и актуальности источников.
3. Сравнительный анализ — выявление общих и противоречивых моментов в разных работах.

Преимущества:

- позволяет опереться на проверенные знания,
- минимизирует субъективность.

Недостатки:

- информация может быть устаревшей,
- требует временного ресурса на поиск и анализ.

Пример.

При разработке медицинской информационной системы анализируются клинические рекомендации, законодательство и научные работы по теме.

Интервью.

Целенаправленная беседа с экспертами или заинтересованными лицами для выявления ключевых аспектов предметной области.

Виды:

1. Структурированное (анкетирование) — строгий список вопросов.
2. Полуструктурированное — гибкий сценарий с возможностью уточнений.
3. Неструктурированное (свободное) — открытая беседа.

Этапы проведения:

1. Подготовка (определение целей, выбор экспертов, составление списка вопросов).
2. Проведение (запись, конспектирование).
3. Анализ.

Преимущества:

- позволяет получить глубокие и детализированные данные,
- выявляет скрытые знания, которые могут быть не описаны в литературе.

Недостатки:

- результаты зависят от компетентности интервьюера и респондента,

- возможна субъективность эксперта.

Пример.

Интервью с бизнес-аналитиком для выявления требований к новой CRM-системе.

Дискуссии.

Коллективное обсуждение проблемы с участием нескольких специалистов для уточнения деталей и поиска решений.

Формы дискуссий:

1. Фокус-группы — обсуждение с целевой аудиторией.
2. Экспертные панели — обмен мнениями между профессионалами.
3. Дебаты — аргументированный спор для выявления сильных и слабых сторон.

Преимущества:

- позволяет рассмотреть проблему с разных точек зрения,
- позволяет выявить противоречия и найти компромиссы.

Недостатки:

- активные участники могут доминировать и “продавливать” свою позицию,
- есть риск ухода от темы.

Пример.

Обсуждение с заказчиком и разработчиками функционала нового мобильного приложения.

Круглые столы.

Организованная встреча экспертов для обсуждения сложных вопросов в формате равноправного диалога.

Преимущества:

- объединяет мнения разных специалистов,
- полезен для междисциплинарных проектов.

Недостатки:

- может затягиваться из-за большого количества участников.

Пример.

Круглый стол с врачами, IT-специалистами и юристами по вопросам цифровизации здравоохранения.

2 - Выделение важнейших компонентов текста.

См. вопрос [12](#)

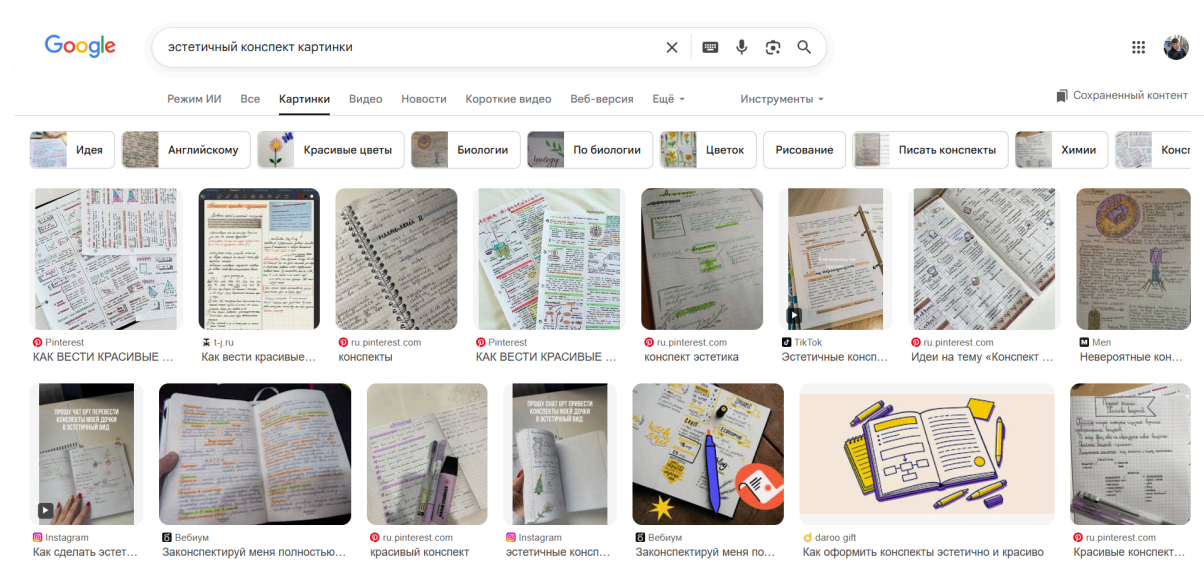
Способы структурирования текста:

- 1) Структурирование информации:
 - определение предметной области,
 - основные понятия и термины,
 - цели и задачи,
 - участники и stakeholders,
 - процессы и взаимосвязи,
 - ограничения и проблемы.
- 2) Использование визуального выделения.
- 3) Ранжирование информации по важности.
- 4) Удаление избыточности:
 - убрать повторяющиеся фрагменты,
 - заменить длинные описания на таблицы/схемы,
 - использовать четкие определения.
- 5) Проверка на полноту и непротиворечивость:
 - все важные элементы должны быть уточнены,
 - не должно быть логических противоречий,
 - описание должно быть понятно даже неспециалисту.

Вопросы для проверки.

- Можно ли на основе этого текста разработать систему?
- Все ли термины объяснены и ясны?
- Есть ли пробелы в описании?

??? что за вопрос вообще



3 - Верификация текста.

Верификация текста — это процесс проверки его на соответствие исходным требованиям, стандартам, фактам и ожиданиям заказчика.

Основные аспекты верификации текста.

1. Проверка на соответствие требованиям.
 - a. Соответствует ли текст ТЗ?
 - b. Учтены ли все ключевые пункты (стиль, структура)?
2. Фактологическая проверка.
 - a. Верны ли приведенные данные: цифры, даты, имена?
 - b. Нет ли логических противоречий?
3. Лингвистическая и стилистическая проверка.
 - a. Проверка корректности грамматики, орфографии, пунктуации.
 - b. Проверка соответствия стилю (научный, официальный, публицистический и т.д.).
4. Проверка на уникальность.
 - a. Использование только оригинальных формулировок.
 - b. Правильность оформления цитат.
5. Экспертная оценка — проверка специалистом предметной области.

Методы верификации текста.

1. Автоматизированные инструменты.
 - a. Орфографические корректоры (MS Word).
 - b. Системы проверки уникальности (Antiplagiat).
 - c. Программы анализа стиля (Главред).
2. Ручная проверка.
 - a. Читка редактором или корректором.
 - b. Сравнение текста с исходными требованиями.
3. Рецензирование экспертами.
 - a. Оценка содержания специалистами данной предметной области.
4. Тестирование на целевой аудитории.
 - a. Понятен ли текст читателям?

4 - Текстовое описание сущностей, атрибутов, значений атрибутов.

Сущность (Entity) — это реальный или абстрактный объект предметной области, информация о котором должна сохраняться и быть доступна в системе.

Атрибут — поименованная характеристика сущности, определяющая её свойства и принимающая значения из некоторого множества (домена). Каждый атрибут имеет имя, уникальное в пределах сущности: `<имя_сущности>.<имя_атрибута>.`

Пример. Студент_Фамилия, Студент_КурсОбучения.

Домен атрибута — множество допустимых значений, которые может принимать атрибут.

Структура текстового описания сущности.

При концептуальном проектировании текстовое описание сущности включает:

1. Описание — краткую семантическую характеристику.
2. Ожидаемый объем экземпляров.
3. Описание атрибутов:
 - a. Имя атрибута.
 - b. Домен атрибута.
 - c. Обязательность атрибута.
 - d. Ключ [PK / FK / -].
 - e. Назначение атрибута.
4. Первичный ключ — список атрибутов-идентификаторов.

Классификация атрибутов.

- **Описательный:** характеризует внутренние свойства экземпляра (Кошка.Вес).
- **Указывающий:** присваивает имя или обозначение экземпляру (Счёт.Номер, Город.Название, Адрес.Индекс).
- **Вспомогательный:** связывает экземпляры одной сущности с экземплярами другой (Кошка.Имя_хозяина).

Для дискретных атрибутов (с конечным множеством значений) значения фиксируются в словаре данных:

Атрибут: Статус_заказа

Домен: { "Новый", "В_обработке", "Отправлен", "Доставлен", "Отменён" }

Значение по умолчанию: "Новый"

Для непрерывных атрибутов указывается тип данных и ограничения:

Атрибут: Цена_товара

Тип: DECIMAL(10,2)

Домен: [0.01 .. 999999.99]

Обязательный: Да

Этапы документирования в словаре данных.

1. Определение сущностей — присвоение им понятных имён и описаний.
2. Определение связей — документирование типов связей (1:1, 1:M, M:N) и классов принадлежности.
3. Определение атрибутов — для каждого атрибута указывается:
 - a. имя и краткое описание,
 - b. тип (описательный/указывающий/вспомогательный),
 - c. домен и ограничения,
 - d. обязательность значения,
 - e. участие в ключах.
4. Фиксация дискретных значений — перечисление допустимых значений для атрибутов с конечным доменом.
5. Определение первичных ключей — выбор минимального набора атрибутов для уникальной идентификации.

5 - Текстовое описание процессов, участников и действий.

Процесс — упорядоченная последовательность действий, выполняемых для достижения определённой цели в рамках системы. В контексте проектирования приложений процесс описывает *что, кем и в каком порядке* выполняется для получения результата.

Участник (актор, роль) — субъект, взаимодействующий с системой: человек, внешняя система, устройство или временной триггер. Акторы делятся на:

- первичный: иницирует процесс,
- вторичный: подключается в ходе процесса, обеспечивает сервис,
- временной: запускает процесс по расписанию/событию.

Действие — элементарная единица процесса, выполняемая участником или системой, имеющая чёткий вход, выход и семантику.

Структура текстового описания процесса.

Стандартизированного шаблона не существует, но общепринятая структура детального описания включает:

1. Цель процесса: краткая формулировка результата процесса.
2. Перечисление участников процесса.
3. Предусловия (например, “пользователь авторизован в системе”, “товар есть в наличии на складе”).
4. Триггеры (например, “пользователь обратился за оформлением заказа”).
5. Основной поток событий (например, утверждение адреса и сроков доставки при заказе товара, оплата и т.д.).
6. Альтернативные потоки (например, недостаток средств на карте -> повторная попытка оплаты, или отсутствие товара на складе -> предзаказ).
7. Исключения и обработка ошибок.
8. Постусловия (определения успеха и неуспеха).
9. Бизнес-правила и ограничения (например, минимальная сумма заказа).

Правила описания действий:

- *атомарность действия* — каждый шаг выполняет одну логическую операцию;
- *однозначность формулировок*;
- *полнота условий* — для каждого ветвления указывать условие перехода;
- *независимость от реализации* — на этапе функционального проектирования описывать что должно происходить, а не как технически это реализовать.

6 - ER-диаграмма. Сущности и отношения.

ER-диаграмма (Entity-Relationship Diagram) — графическая модель данных, которая отображает сущности, их атрибуты и связи между ними.

Основные элементы ER-диаграммы

1. **Сущности (Entities)** — объекты предметной области, о которых необходимо хранить информацию (например, **Питомец**).
 - Обозначается *прямоугольником* с названием сущности.
2. **Атрибуты (Attributes)** — характеристики сущности (например, для Питомца: **кличка**, **вид животного**, **пол**, **вес**).
 - Типы атрибутов:
 - простые, неделимые (например, **возраст**),
 - составные (например, **адрес** -> **город**, **улица**, **дом**, **квартира**),
 - ключевые (уникальные идентификаторы, типа **ID**),
 - производные (те, которые можно вычислить, например, **возраст** по дате рождения).
 - Обозначается *овалом*, соединенным с сущностью.
3. **Связи (Relationships)** — взаимодействия между сущностями (например, **Врач** проводит **Приём**).
 - Степени связи:
 - 1 : 1 — Паспорт : Человек,
 - 1 : N — Автор : Книги,
 - N : N — Студенты : Курсы.
 - Обозначается *ромбом* с линиями к сущностям.

Типы ER-диаграмм

1. *Концептуальная модель* — высокоуровневое описание сущностей и их связей без технических подробностей. Используется на этапе анализа требований.
2. *Логическая модель* — детализированное описание с указанием атрибутов, типов данных и ключей. Является подготовкой к реализации физической модели.
3. *Физическая модель* — готовая схема для конкретной СУБД, которая включает индексы, ограничения и оптимизации.

7 - ER-диаграмма. Типы связей сущностей.

ER-диаграмма (Entity-Relationship Diagram) — графическая модель данных, которая отображает сущности, их атрибуты и связи между ними.

Типы связей между сущностями в ER-диаграммах

1. Один к одному (1 : 1)

- a. Одна запись в сущности А связана ровно с одной записью в сущности Б.
- b. Обозначается одинарной линией на обоих концах связи.
- c. Пример: Человек и Паспорт.

2. Один ко многим (1 : N)

- a. Одна запись в сущности А может быть связана с несколькими записями в сущности Б.
- b. Обозначение: линия с обозначениями “1” и “N” на концах/”вилка” на стороне “многих”, одинарная линия на стороне “одного”.
- c. Пример: Автор и Книги.

3. Многие ко многим (M : N)

- a. Несколько записей в сущности А могут быть связаны с несколькими записями в сущности Б.
- b. Обозначение: линия с обозначениями “M” и “N” на концах/с “вилками” на обоих концах.
- c. Пример: Студенты и Курсы.

Для связи “многие ко многим” в реляционных БД требуется промежуточная таблица “сущность – связь”:

Студенты (ID, Имя)

Курсы (ID, Название)

-> Посещения (ID_Студент, ID_Курс)

Другие характеристики связей

1. Обязательность.

- a. Сплошная линия — обязательная связь (у Заказа обязательно есть Покупатель).
- b. Пунктирная линия — необязательная связь (не у каждого Клиента обязательно должен быть Заказ).

2. Мощность связи.

- a. Указывает на минимальное/максимальное количество связей. Например, (1, 1) – ровно 1 связь, (0, N) – от нуля до многих.

8 - ER-диаграмма. Атрибуты сущностей. Сильные и слабые сущности.

ER-диаграмма (Entity-Relationship Diagram) — графическая модель данных, которая отображает сущности, их атрибуты и связи между ними.

Атрибут — свойство сущности, принимающее значения из определенного домена. На ER-диаграмме атрибуты отображаются овалами, соединенными линиями с прямоугольником сущности.

Классификация атрибутов

1. По структуре

- a. Простой (атомарный) — не делится на части (например, [Дата_Рождения](#)).
- b. Составной — состоит из податрибутов (например, [Адрес](#) -> [Город](#), [Улица](#), [Дом](#), [Квартира](#)).
- c. Многозначный — может принимать несколько значений для одного экземпляра сущности (например, [Телефоны](#) для [Клиента](#)).
- d. Вычисляемый — значение определяется по формуле (например, [Возраст](#), [Сумма_заказа](#)).

2. По роли в идентификации

- a. Ключевой — входит в первичный ключ. Обозначается как подчеркнутый атрибут.
 - i. Простой ключ: [ID](#).
 - ii. Составной ключ: [Номер_зачетки](#) + [Год_поступления](#).
- b. Неключевой — описывает свойства и не участвует в идентификации.

3. По обязательности

- a. Обязательный — NOT NULL.
- b. Необязательный — разрешен NULL.

Правила проектирования атрибутов

1. **Атомарность:** в концептуальной модели атрибут не должен содержать повторяющихся или составных значений (требование 1НФ).
2. **Уникальность имен:** в пределах одной сущности имена атрибутов не повторяются.
3. **Семантическая самостоятельность:** атрибут описывает сущность целиком, а не ее часть или другую сущность.
4. **Доменная определенность:** для каждого атрибута фиксируется тип данных, диапазон значений и формат.

Слабые и сильные сущности

1. Сильная сущность

- a. Существует самостоятельно, не зависит от других сущностей.
- b. Имеет собственный первичный ключ.

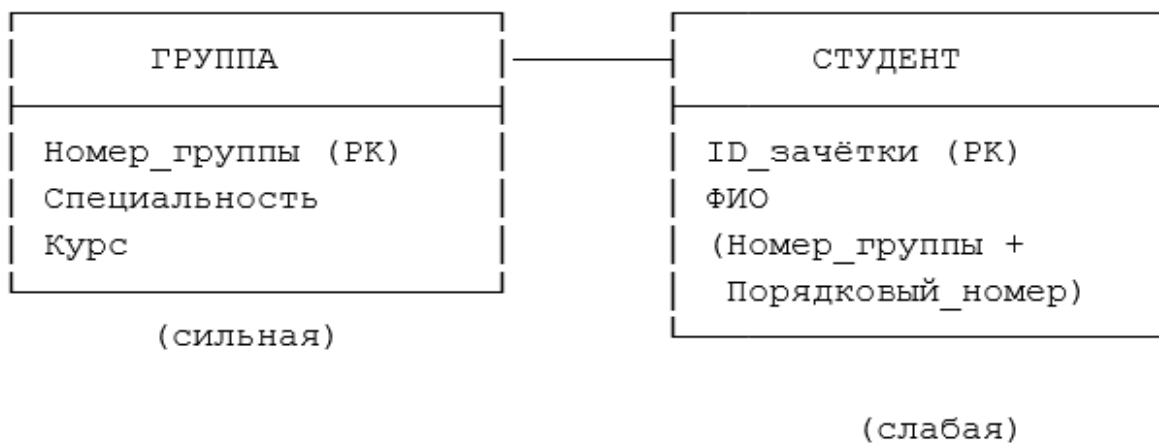
- c. Удаление происходит независимо от других сущностей. При удалении сильной сущности удаляются все зависимые от нее сущности.
- d. Связи любые: (1 : 1), (1 : N), (M : N).

2. Слабая сущность

- a. Зависит от существования сущности-владельца.
- b. Не имеет собственного РК, идентифицируется через РК_владельца + дискриминатор.
- c. Каскадное удаление при удалении владельца.
- d. Связь только (1 : M) со стороны владельца (один владелец -> много слабых).
- e. Обозначается двойным прямоугольником (в нотации Чена) или пунктиром.

Дискриминатор — атрибут (или набор атрибутов) слабой сущности, который уникален только в контексте одного экземпляра сущности-владельца.

Пример: **Номер_Рейса** уникален только в рамках одного Автобуса. Вместе они образуют составной ключ (**ID_Автобуса**, **Номер_рейса**).



9 - ER-диаграмма. Ключевые атрибуты. Простые и составные ключевые атрибуты.

ER-диаграмма (Entity-Relationship Diagram) — графическая модель данных, которая отображает сущности, их атрибуты и связи между ними.

Ключевой атрибут — атрибут (или набор атрибутов), значения которого однозначно идентифицируют каждый экземпляр сущности в рамках моделируемой предметной области. В ER-модели ключевые атрибуты формируют первичный ключ (Primary Key, PK) сущности.

Функции ключевого атрибута:

- обеспечение целостности сущности (запрет дубликатов);
- основа для построения связей (внешние ключи в связанных сущностях ссылаются на PK);
- точка входа для поиска, обновления и удаления записи.

В ER-модели различают:

1. Потенциальный (Candidate Key) — минимальный набор атрибутов, гарантирующий уникальность записи.
2. Первичный (Primary Key) — ключ, выбранный из потенциальных для физической реализации.
3. Внешний (Foreign Key) — атрибут, ссылающийся на PK другой сущности.

Простые ключевые атрибуты

Простой ключ — первичный ключ, состоящий из *одного* атрибута. Подчеркивается на ER-диаграмме.

Характеристики:

- типы:
 - естественный (имеет бизнес-смысл): ИНН,
 - суррогатный (системный идентификатор без смысла): ID.
- преимущества:
 - простота FK-ссылок,
 - высокая производительность JOIN,
 - минимальный объем индекса.
- недостатки:
 - естественные ключи могут меняться (например, смена email),
 - суррогатные ключи не несут бизнес-информации.
- примеры: **ID_студента**, **Номер_паспорта**, **Email**.

Составные ключевые атрибуты

Составной ключ — первичный ключ, образованный двумя или более атрибутами, которые в комбинации гарантируют уникальность экземпляра, но по отдельности — нет.

Характеристики:

- минимальность: ни одно подмножество атрибутов составного ключа не должно быть уникальным;
- обязательность: все атрибуты ключа должны быть NOT NULL;
- применяется для слабых сущностей, для ассоциативных сущностей (связь M : N после декомпозиции);
- преимущества:
 - отражают бизнес-логику,
 - исключают дублирование в контексте;
- недостатки:
 - сложные FK-ссылки,
 - большой размер индекса;
- примеры: (Код_факультета, Номер_группы) -> Группа.

Правила выбора и проектирования ключевых атрибутов

1. **Уникальность:** значение не должно повторяться среди экземпляров сущности.
2. **Минимальность:** нельзя удалить ни один атрибут из ключа без потери уникальности.
3. **Стабильность:** значение не должно изменяться в течение жизненного цикла экземпляра (предпочтение суррогатным РК для основных сущностей).
4. **Простота:** по возможности использовать одноатрибутный ключ (упрощает FK, индексы).
5. **Непустота:** NULL недопустим в ключевых атрибутах (реляционная теория и SQL стандарт).
6. **Семантическая нейтральность** (для суррогатных): ключ не должен нести бизнес-смысл, чтобы избежать логических зависимостей.

10 - ER-диаграмма. Бинарные, тернарные и n-арные связи.

ER-диаграмма (Entity-Relationship Diagram) — графическая модель данных, которая отображает сущности, их атрибуты и связи между ними.

Связь (Relationship) — это ассоциация (отношение) между двумя или более сущностями.

Арность (степень) связи — количество сущностей, участвующих в ассоциации. В зависимости от арности связи классифицируются на:

- бинарная: связывает ровно две сущности;
- тернарная: связывает три сущности одновременно;
- n-арная ($n > 3$): связывает n сущностей.

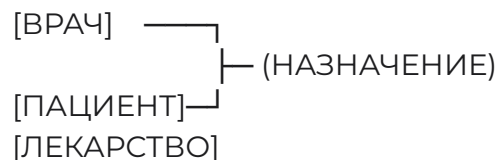
Тернарные связи

Связь между *тремя* сущностями. Возникает, когда бизнес-правило не может быть корректно разложено на две бинарные связи без потери семантики или появления избыточности.

Возникает, когда:

- атрибут связи зависит от *комбинации всех трёх* сущностей,
- разложение на бинарные связи создаёт «висячие» комбинации или допускает логически невозможные состояния.

Пример: «*Конкретный врач назначает конкретное лекарство конкретному пациенту на определённую дату*».



Кардинальность связи указывается относительно **каждой** сущности к связи.

N-арные связи

Обобщение тернарных связей. Связывают *четыре и более* сущностей.

Особенности:

- в практическом проектировании встречаются редко,
- часто являются следствием недостаточной декомпозиции предметной области.

11 - ER-диаграмма. Проблема «единичного значения атрибута». Пропущенные сущности, отношения и атрибуты.

ER-диаграмма (Entity-Relationship Diagram) — графическая модель данных, которая отображает сущности, их атрибуты и связи между ними.

Проблема нарушения правила «единичного значения атрибута»

В корректной ER-модели каждый атрибут сущности должен принимать ровно одно значение для данного экземпляра в конкретный момент времени.

На практике под этим термином понимают **нарушение правила атомарности**, когда атрибут фактически хранит множество значений.

Пример нарушения	Почему это проблема	Корректное решение 
Студент.Номера_телефонов = ["+7900...", "+7901..."]	Невозможно проиндексировать, обновлять отдельно, нарушается 1НФ	Вынести в сущность КОНТАКТ_СТУДЕНТА (1:M)
Товар.Размеры = ["S", "M", "L"]	Дублирование строк при запросах, сложность фильтрации	Связь M:N через НАЛИЧИЕ_НА_СКЛАДЕ или отдельный справочник РАЗМЕР
Сотрудник.Навыки = ["SQL", "Python", "Agile"]	Аномалии обновления, потеря истории изменений	Сущность НАВЫК_СОТРУДНИКА с датой подтверждения и уровнем

Последствия игнорирования проблемы:

- нарушение первой нормальной формы (1НФ) на логическом уровне,
- аномалии вставки, обновления и удаления данных,
- сложность построения JOIN-запросов и индексации.

Пропущенные сущности, отношения и атрибуты

Это ошибки *полноты модели*, возникающие на этапе сбора требований и концептуального проектирования. Они приводят к тому, что ER-диаграмма не покрывает все бизнес-сценарии и не поддерживает целостность данных.

Тип пропуска	Пример	Последствия	Как выявить 
Пропущенная сущность	Смоделированы ЗАКАЗ и ТОВАР, но нет ПОСТАВЩИК или ДОСТАВКА	Невозможно отследить логистику, складские остатки, историю поставок	Анализ end-to-end сценариев, интервью с экспертами, CRUD-матрица
Пропущенная связь	Есть ВРАЧ и ПАЦИЕНТ, но нет связи НАЗНАЧАЕТ_ЛЕЧЕНИЕ	Потеря контекста: кто, что и когда назначил	Трассировка Use Case → элементы модели, проверка бизнес-правил
Пропущенный атрибут	У ПОЛЬЗОВАТЕЛЯ нет Email, Даты_регистрации, Статуса_аккаунта	Невозможность аутентификации, аналитики, блокировок	Словарь данных, чек-лист обязательных полей по типу сущности

че

12 - ER-диаграмма. Составление описания по тексту. Проверка согласования диаграммы и исходного текстового описания.

Переход от текстового описания предметной области к ER-диаграмме — *ключевой этап инфологического проектирования*. Его задача: формализовать бизнес-требования, изложенные в естественном языке, в виде структурированной графической модели, готовой к логическому преобразованию в реляционную схему.

Алгоритм составления ER-диаграммы по тексту

1. Лингвистический разбор — выделение значимых слов в тексте:

- существительные -> кандидаты в сущности/атрибуты;
- глаголы/предлоги -> кандидаты в связи;
- прилагательные/уточнения -> домены/ограничения.

В результате получаем список потенциальных элементов модели.

2. Фильтрация и классификация — отбор и категоризация кандидатов:

- удаление синонимов, абстракций, технических деталей;
- проведение теста на атомарность: может ли элемент иметь >1 значения? нет -> вынести в сущность.

В результате получаем четкое разделение на сущности, атрибуты и связи.

3. Определение ключей — выбор идентификаторов:

- правило минимальности и уникальности;
- маркировка PK;
- выделение суррогатных и естественных ключей.

В результате: атрибуты-идентификаторы для каждой сущности.

4. Установление связей — формирование ассоциаций:

- парный анализ сущностей,
- определение типа связей,
- указание кардинальности и обязательности связей.

Результат: связи с семантическими метками.

5. Визуализация — строим диаграмму.

Правило-фильтр: если понятие в тексте описывает объект, имеющий собственную жизнь, атрибуты или связи → это **сущность**. Если лишь характеризует объект → **атрибут**.

Критерии успешного согласования

1. *Полнота:* каждый значимый объект и правило из текста представлен в модели.
2. *Точность:* кардинальности, ключи, типы связей строго соответствуют бизнес-логике.
3. *Минимальная достаточность:* модель не содержит избыточных сущностей и связей, дублирующих друг друга.

4. *Трассируемость*: для каждого элемента диаграммы можно указать источник в тексте, и наоборот.

Методы проверки согласования

Метод	Описание	Что выявляет 
Обратный перевод	По готовой ER-диаграмме формулируются текстовые правила без использования оригинала. Сравнение с исходным текстом.	Потерю смысла, добавление несуществующих правил, искажение кардинальностей
Матрица трассировки	Строки: фрагменты текста/требования. Столбцы: сущности, связи, атрибуты. Отметки  показывают покрытие.	Пропущенные элементы, избыточные компоненты, «висячие» сущности
Сценарное тестирование	Проход по бизнес-процессам (Use Cases) с проверкой участия всех объектов на каждом шаге.	Неподдерживаемые потоки, отсутствие ключевых атрибутов для фильтрации/состояний
Формальная проверка	Контроль по правилам ER-моделирования: атомарность, уникальность РК, корректность  , отсутствие циклических зависимостей без необходимости.	Нарушения нормализации, логические противоречия, риски аномалий
Экспертная верификация	Рецензирование модели предметным экспертом, аналитиком, архитектором.	Скрытые бизнес-правила, неочевидные зависимости, отраслевые нюансы

13 - Цели описания ситуативного взаимодействия.

Ситуативное взаимодействие — это описание взаимодействия участников системы (пользователей, внешних систем, сотрудников организации) в рамках определённой ситуации или сценария.

Обычно такое описание используется при анализе бизнес-процессов для понимания:

- кто участвует во взаимодействии;
- какие действия выполняются;
- в какой последовательности происходят события;
- какие данные используются;
- какой результат должен быть получен.

На основе описания ситуативного взаимодействия строятся use-case модели, BPMN-диаграммы, описания бизнес-процессов, функциональные требования к системе.

Основные цели

1. Понимание предметной области
Например, как работает организация, какие действия выполняют участники процессов, какие сценарии процесса существуют и тд
2. Выявление требований к системе
Ситуативное взаимодействие позволяет определить:
 - необходимые функции системы;
 - роли пользователей;
 - ограничения;
 - входные и выходные данные;
 - реакции системы на действия пользователя.
3. Определение участников процесса
4. Формализация бизнес-процессов
5. Обнаружение проблем и узких мест
При анализе взаимодействий можно выявить:
 - лишние действия;
 - дублирование операций;
 - неэффективные этапы процесса.
6. Подготовка к автоматизации процессов (Попов не особо любит в связи с анализом процессов про автоматизацию говорить, так что не факт, что это стоит писать)
 - какие операции можно автоматизировать;
 - какие действия должны выполнять пользователи;

- *какие данные должна хранить система.*

Элементы описания ситуативного взаимодействия

- название сценария;
- цель сценария;
- участников;
- входные данные;
- триггеры;
- основной поток событий;
- альтернативные потоки;
- результат выполнения.

14 - Изолированность ситуативного взаимодействия.

Изолированность ситуативного взаимодействия — это свойство, при котором выполнение одного взаимодействия рассматривается как отдельный независимый процесс, не влияющий напрямую на другие взаимодействия.

Основная идея изолированности состоит в том, чтобы во время выполнения одного сценария:

- другие процессы не должны нарушать его состояние;
- изменения должны быть согласованными;
- участники работают только со своей частью процесса.

Изолированность необходима для:

- предотвращения конфликтов между процессами;
- обеспечения корректности выполнения действий;
- сохранения целостности данных;
- независимого выполнения сценариев;
- повышения надёжности процессов.

Пример: бронирование места

Если два клиента одновременно пытаются забронировать одно место, система должна обеспечить изолированность:

- место не должно быть выдано двум клиентам одновременно;
- действия одного клиента не должны нарушать процесс другого.

Проблемы при отсутствии изолированности

- конфликты данных;
- потеря данных;
- нарушение логики процесса;
- несогласованность результатов.

Средства обеспечения изолированности

- разграничение доступа (прав пользователей в системах);
- блокировка ресурсов (во время выполнения процесса ресурс временно становится недоступным для других операций);
- транзакции (изменения выполняются как единое целое).

15 - Сценарии использования (use case). Перечень разделов описания сценариев использования.

Сценарий использования (Use Case) — это описание последовательности действий участников для достижения определённой цели в рамках некоторого процесса или ситуации.

Сценарий показывает:

- кто участвует во взаимодействии;
- какие действия выполняются;
- в какой последовательности происходят события;
- какой результат должен быть получен.

Юз-кейсы основываются на том, что любое действие совершает человек, даже если что-то автоматически включилось, то источником этого действия все равно являлся человек. Все механизмы — это лишь средства/инструменты. То есть важна функция, а не реализация.

Основные элементы описания использования

1. название;
2. участники (акторы);
*компьютер, система, программа не являются акторами
**иногда добавляют выгодоприобретателя
3. цель;
4. входные данные;
5. триггер;
6. основной сценарий/процесс;
7. альтернативные сценарии/процессы;
8. результаты.

16 - Позитивные и негативные исходы use case.

Сценарий использования (Use Case) описывает последовательность действий участников процесса для достижения определённой цели.

Исход Use Case — это итог выполнения сценария, определяющий, была ли достигнута поставленная цель.

Основными видами исходов являются:

- позитивный исход;
- негативный исход.

Позитивный исход

Позитивный исход — это успешное завершение сценария, при котором цель процесса достигнута.

При позитивном исходе:

- все необходимые действия выполнены;
- ограничения соблюдены;
- результат получен;
- процесс завершён корректно.

Позитивный исход обычно соответствует основному потоку сценария.

Негативный исход

Негативный исход — это завершение сценария без достижения основной цели.

Он возникает при:

- ошибках;
- нарушении условий;
- отказе пользователя;
- невозможности продолжения процесса.

Негативный исход обычно связан с альтернативными или исключительными потоками сценария.

Пример 1: оформление заказа

Сценарий:

1. Клиент выбирает товар.
2. Менеджер оформляет заказ.
3. Клиент оплачивает покупку.
4. Заказ подтверждается.

Позитивный исход: Заказ успешно оформлен и оплачен.

Негативный исход: оплата не проходит

Пример 2: бронирование места

Сценарий:

1. Клиент выбирает дату.
2. Администратор проверяет наличие мест.
3. Оформляется бронирование.

Позитивный исход: Место успешно забронировано.

Негативный исход: Свободных мест нет, бронирование невозможно.

17 - Основной и альтернативный сценарий use case.

Сценарий использования (Use Case) — это описание последовательности действий участников процесса для достижения определённой цели.

В рамках одного Use Case обычно выделяют:

- основной сценарий;
- альтернативные сценарии.

Основной сценарий

Основной сценарий — это описание стандартного успешного выполнения процесса, при котором цель достигается без ошибок и отклонений.

Основной сценарий:

- описывает успешное выполнение процесса;
- не включает ошибки и исключения;
- отражает наиболее вероятный вариант выполнения.

Альтернативный сценарий

Альтернативный сценарий — это вариант выполнения процесса, отличающийся от основного, но всё ещё приводящий к продолжению или завершению процесса.

Альтернативный сценарий описывает дополнительные варианты действий или другие способы достижения результата. Он возникает в ситуациях, когда процесс развивается не по основному пути.

Альтернативный сценарий:

- дополняет основной;
- возникает на определенном шаге процесса;
- изменяет часть последовательности действий;
- может завершаться как успешно, так и неуспешно.

Пример 1: оплата покупки

Основной сценарий: оплата картой

Альтернативный сценарий: оплата наличными, оплата купоном, оплата подарочной картой

Пример 2: бронирование места

Основной сценарий:

1. Клиент выбирает дату.
2. Администратор проверяет наличие мест.
3. Оформляется бронирование.

4. Клиент получает подтверждение.

Альтернативный сценарий:

Если мест нет:

1. Администратор предлагает другую дату.
2. Клиент выбирает новый вариант.
3. Бронирование продолжается.

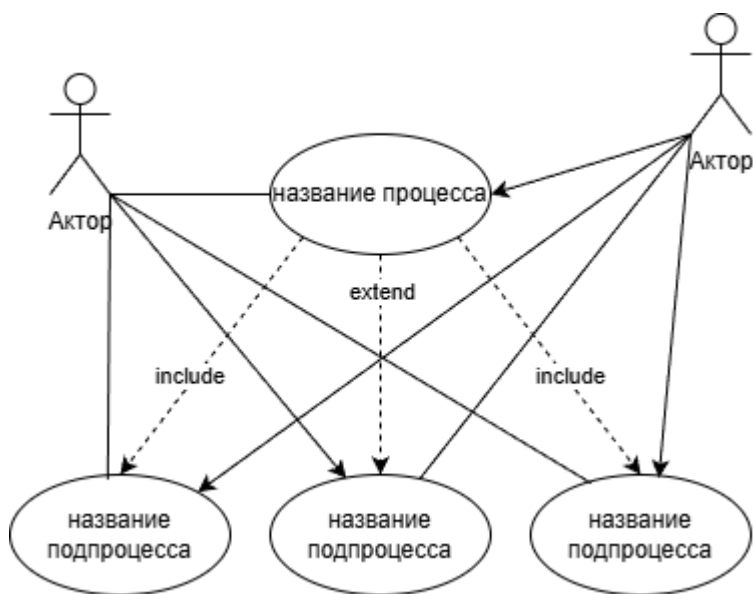
18 - Диаграммы сценариев использования. Синтаксис графического представления. Действия и ассоциации.

Диаграмма сценариев использования (Use Case Diagram) — это графическое представление взаимодействия участников с процессами или функциями предметной области.

Диаграмма показывает:

- участников взаимодействия;
- выполняемые действия;
- связи между участниками и действиями;
- структуру взаимодействий.

Use Case диаграммы относятся к языку моделирования UML (Unified Modeling Language).



стрелка означает, что актер инициирует данный процесс

На первом уровне должны быть все акторы, даже если они участвуют только в более мелких подчиненных юз-кейсах.

Основные элементы:

- Актор – человек;
- Сценарий использования/use-case/процесс – эллипс;
- Ассоциация – сплошная линия;
- Include – пунктирная стрелка с "include";
- Extend – пунктирная стрелка с "extend";
- Обобщение – стрелка с пустым треугольником.

Ассоциации

Ассоциация — это связь между актором и сценарием использования.

Она показывает, что участник:

- инициирует действие;
- участвует в выполнении процесса;
- взаимодействует с процессом.

19 - Ассоциации, обобщения, включения и расширения.

В диаграммах сценариев использования (Use Case Diagram) применяются различные типы связей между элементами модели.

Связи позволяют показать:

- взаимодействие участников с действиями;
- зависимость между сценариями;
- общие и специализированные элементы;
- дополнительные варианты поведения.

Основные типы связей

- ассоциация (association);
- обобщение (generalization);
- включение (include);
- расширение (extend).

Ассоциации

Ассоциация — это связь между актором и сценарием использования, показывающая их взаимодействие.

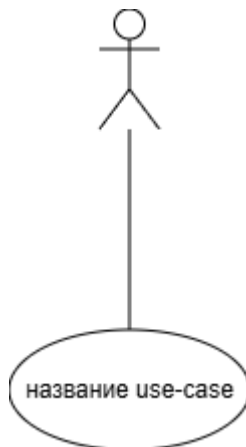
Она означает, что:

- участник инициирует действие;
- участвует в выполнении процесса;
- взаимодействует с определённым сценарием.

Ассоциация используется для:

- указания участников процесса;
- отображения взаимодействий;
- определения ролей акторов.

Обозначается сплошной линией между актором и названием юз кейса.



Обобщение

Обобщение — это отношение наследования между элементами модели.

Оно показывает, что:

- один элемент является более общим;
- другой – специализированным вариантом.

Обобщение может применяться:

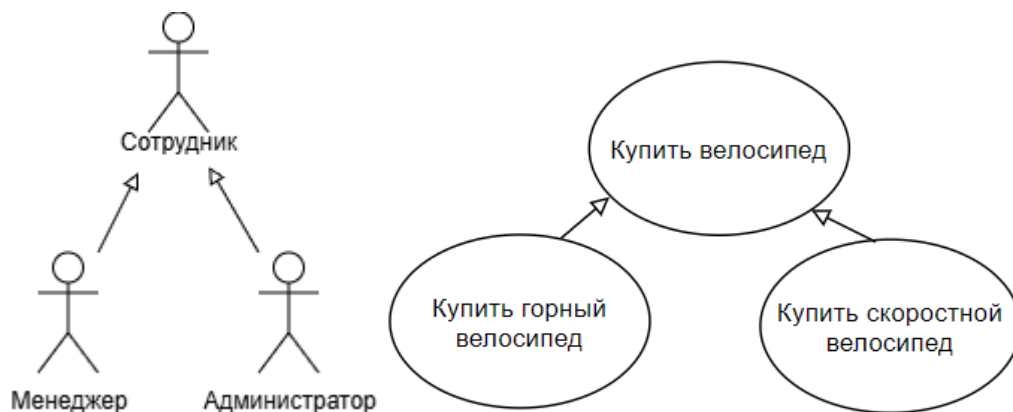
- между акторами;
- между сценариями использования.

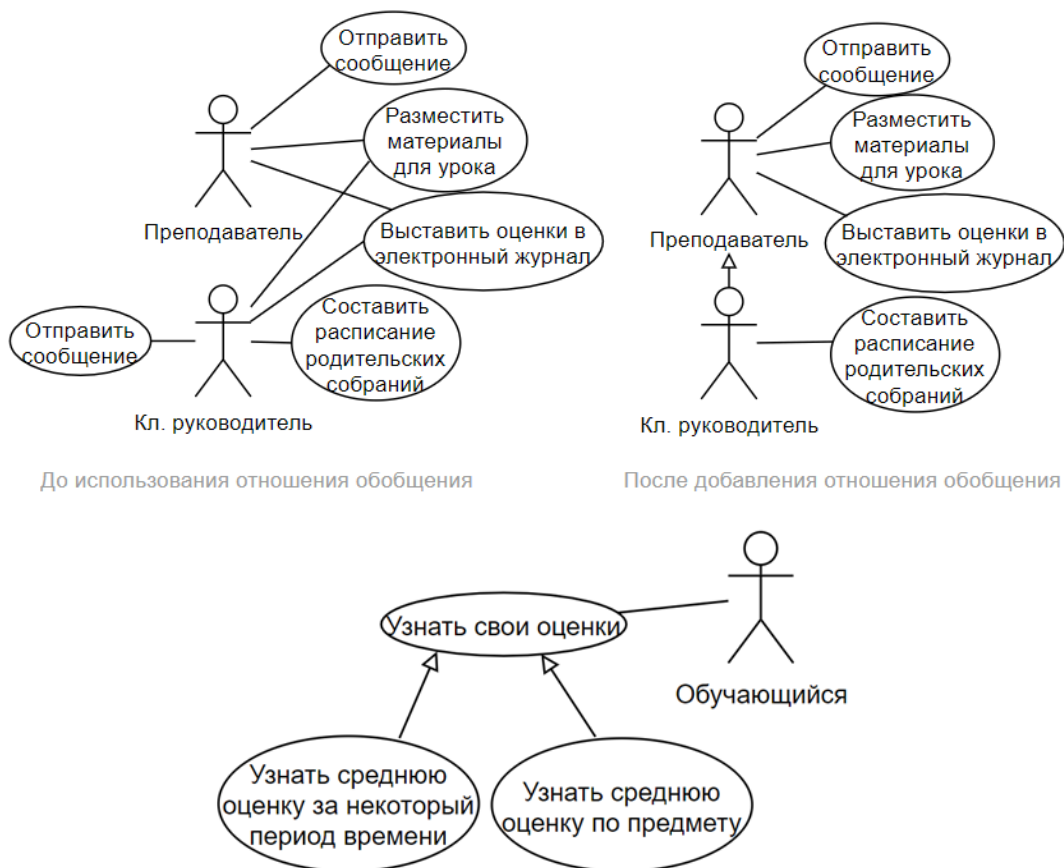
Обобщение используется для:

- уменьшения дублирования;
- выделения общих свойств;
- построения иерархий;
- упрощения модели.

Обозначается сплошной линией с пустой стрелкой, направленной в сторону более общего элемента. (хз правильно ли на картинке, попов такого не выдумывал)

похоже это работает как наследование классов в ооп, типо специализированные варианты наследуют свойства общего





Включение

Включение — это связь, при которой один сценарий обязательно включает выполнение другого сценария.

Связь include используется, если:

- часть действий повторяется;
- общий фрагмент используется в нескольких сценариях.

Включаемый сценарий выполняется всегда.

Используется для выделения общей логики и упрощения диаграммы.

Обозначается пунктирной линией со стрелкой, направленной к включаемому сценарию, и подписью "include".



Расширение

Расширение — это связь, при которой один сценарий дополняет другой дополнительным или необязательным поведением.

Связь extend используется для:

- описания необязательных действий;
- обработки альтернативных ситуаций.

Расширение выполняется не всегда.

Обозначается пунктирной линией со стрелкой, направленной к включаемому действию, и подписью “extend”.



20 - Акторы. Отображение обобщения акторов.

Актор — это участник взаимодействия, который выполняет определённые действия или участвует в процессе.

Актор взаимодействует со сценариями использования (Use Case) и инициирует выполнение действий.

Актор представляет не конкретного человека, а роль, которую участник выполняет в процессе.

Актором может быть человек, группа людей, организация. Компьютер, система, программа не являются акторами.

На use-case диаграмме обозначается человечком с подписью роли.

Обобщение акторов

Обобщение акторов (Generalization) — это отношение наследования между акторами, при котором один актор является более общим, а другой — его специализированной разновидностью.

Специализированный актор:

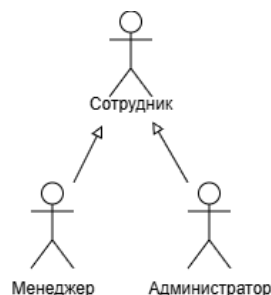
- наследует свойства общего актора;
- может выполнять все его действия;
- может иметь дополнительные функции.

Обобщение используется для:

- выделения общих свойств;
- уменьшения дублирования;
- построения иерархий ролей;
- упрощения диаграмм.

Если специализированный актор связан с общим актором через обобщение, то он наследует все связи общего актора и может участвовать во всех его сценариях использования.

Обозначается сплошной линией с пустой стрелкой, направленной от специализированного актора к более общему актору.



21 - Include и extend. Способы использования и ключевые операции.

Include и extend — это отношения между сценариями.

Include

Include — это отношение, при котором один сценарий использования обязательно включает выполнение другого сценария.

Иными словами часть действий выносится в отдельный сценарий, этот сценарий всегда выполняется как часть основного.

Include используется для:

- выделения повторяющихся действий;
- упрощения диаграммы;
- модульного описания процессов (выделения общей логики).

Обозначается пунктирной линией со стрелкой, направленной к включаемому сценарию, и подписью “include”.



Пример: сценарии <<оформить кредит>> и <<открыть счет>> оба включают <<проверить документы>>.

Extend

Extend — это отношение, при котором один сценарий расширяет другой дополнительным или необязательным поведением.

Расширение выполняется не всегда.

Extend используется для:

- описания необязательных действий;
- обработки альтернативных ситуаций

Обозначается пунктирной линией со стрелкой, направленной к включаемому действию, и подписью “extend”.



Пример: сценарий «оформление заказа» может включать «применить скидку».

22 - Плоская и иерархическая нотация в представлении use case.

При построении сценариев использования (Use Case) могут применяться разные способы организации и представления сценариев.

Основными вариантами являются плоская нотация и иерархическая нотация.

Плоская нотация

Плоская нотация — это способ представления Use Case, при котором все сценарии использования располагаются на одном уровне без выделения иерархии и подчинённости.

Каждый сценарий рассматривается:

- как самостоятельный элемент;
- без деления на уровни;
- без вложенной структуры.

Плоская нотация подходит:

- для небольших проектов;
- для простых процессов;
- для начального этапа анализа;
- для быстрого описания взаимодействий.

Пример: отдельные сценарии (отображаются независимо)

- «Проверить данные клиента»;
- «Оформить заказ»;
- «Рассчитать стоимость».

Иерархическая нотация

Иерархическая нотация — это способ представления Use Case, при котором сценарии использования организуются в виде структуры с уровнями и подчинённостью.

Сценарии могут:

- включать другие сценарии;
- расширять друг друга;
- наследовать общие свойства.

В иерархической нотации используются include, extend и generalization, а также выделяются основные и вспомогательные сценарии.

Иерархическая модель применяется:

- в крупных проектах;
- в сложных бизнес-процессах;

- при наличии повторяющихся действий;
- при необходимости детализации процессов.

Пример: сценарий «Оформить заказ» включает: «Проверить данные клиента», «Рассчитать стоимость».

коала ебашит

коала устала

коале надо отдохнуть....

коала согласна, жаль отчет для дедуса сам себя не сделает

23 - Понятие бизнес-процесса.

Бизнес-процесс — последовательность операций, имеющая четко определенные начало и конец во времени, направленная на получение заданного результата.

Процесс состоит из следующих элементов::

- Триггер: Событие, которое запускает процесс. *Пример: Клиент заполнил форму на сайте.*
- Входные данные: ресурсы (материальные, информационные), необходимые для выполнения и получения результата процесса, которые потребляются или преобразовываются при выполнении процесса
- Шаги достижения цели.
- Исполнители (роли).
- Выход (результат): цель выполнения процесса. *Пример: Товар доставлен, деньги на счету.*

Для чего нужны бизнес-процессы:

Понятие бизнес-процесс лежит в основе процессного подхода к анализу и синтезу деятельности организации. Процессный подход позволяет рассматривать деятельность организации как связанную систему бизнес-процессов, каждый из которых протекает во взаимосвязи с другими бизнес-процессами или внешней средой. В настоящий момент применение процессного подхода является обязательным условием для построения Системы менеджмента качества в соответствии с требованиями стандарта ISO 9001. Практика показывает, что система управления, построенная на принципах процессного управления, является более эффективной и результативной по сравнению с равной ей по масштабу функциональной системой.

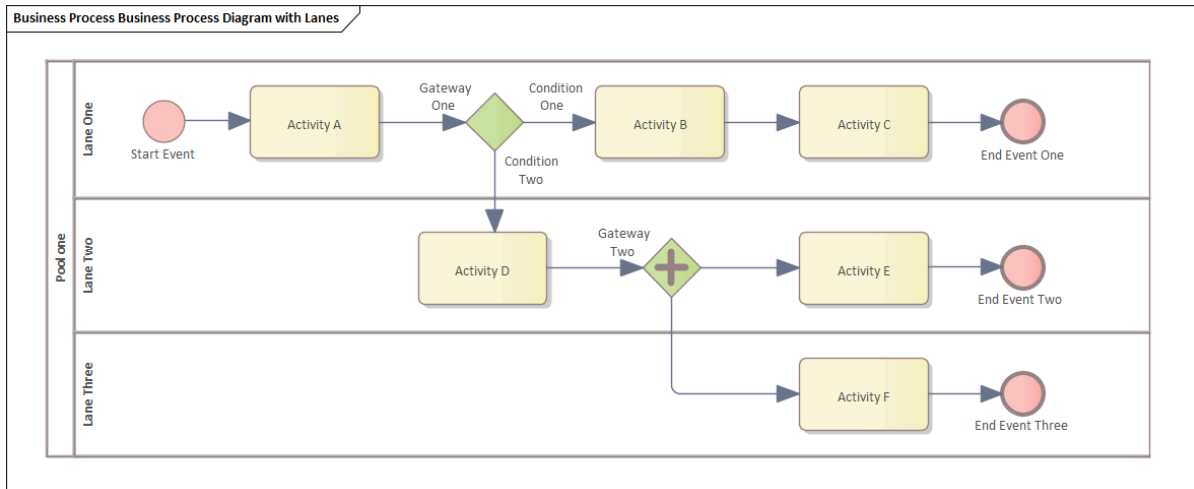
Пример бизнес-процесса:

[Покупатель оформил заказ] -> (Проверка на складе) -> (Оплата) -> (Сборка) -> (Доставка) -> [Товар у клиента]

1. Вход: Данные заказа и корзины.
2. Шаг 1: Система автоматически проверяет остатки на складе.
3. Шаг 2: Бухгалтерия (или платежный шлюз) фиксирует оплату.
4. Шаг 3: Сотрудник склада собирает коробку.
5. Шаг 4: Курьерская служба доставляет посылку.

6. Выход: Довольный клиент и прибыль компании.

Наиболее популярным способом представления бизнес-процесса являются BPMN диаграммы.



24 - Основные и вспомогательные бизнес-процессы.

Бизнес-процесс — последовательность операций, имеющая четко определенные начало и конец во времени, направленная на получение заданного результата.

По отношению к добавленной стоимости процессы делят на *основные* и *вспомогательные* процессы.

Основными бизнес-процессами являются процессы, ориентированные на производство товара или оказание услуги, являющиеся целевыми объектами создания предприятия и обеспечивающие получение дохода. Так, для завода древесно-стружечных плит и деталей основным бизнес-процессом является производство ламинированной древесно-стружечной плиты.

Примеры: производство, продажа, сервисное обслуживание.

Вспомогательные бизнес-процессы не создают добавленной стоимости продукта, предлагаемого предприятием. Фактически они снабжают ресурсами всю деятельность организации и обеспечивают работу основных процессов. При этом деятельность и персонал вспомогательных процессов не взаимодействуют напрямую с произведенной продукцией и/или услугами.

Примеры: наем рабочих, ведение отчетности.

У бизнес-процесса должен быть единый менеджер, который управляет процессом и отвечает за его результат. В деятельности любой компании можно насчитать как минимум несколько десятков бизнес-процессов.

25 - Повторяемость бизнес-процессов.

1. Определение

Повторяемость бизнес-процессов — способность организации выполнять процессы единообразно при одинаковых входных данных, обеспечивая предсказуемый результат. Это ключевой элемент стандартизации и масштабируемости бизнеса.

2. Ключевые элементы повторяемости

1. Стандартизация

- Четкие инструкции и регламенты для каждого этапа.
- Пример: использование шаблонов документов или скриптов для колл-центра.

2. Автоматизация

- Внедрение IT-решений для исключения ручных ошибок.
- Пример: CRM-система для автоматического создания задач менеджерам.

3. Документирование

- Фиксация всех шагов процесса в доступной форме.
- Пример: чек-листы для сотрудников склада.

3. Преимущества повторяемости

- Снижение ошибок: Минимизация человеческого фактора.
- Повышение эффективности: Сокращение времени на выполнение задач.
- Масштабируемость: Возможность тиражировать процессы в новых филиалах.
- Упрощение контроля: Легче отслеживать KPI и проводить аудиты.

4. Методы обеспечения повторяемости

Метод	Описание
BPMN-моделирование	Визуальное описание процессов (например, в нотации BPMN).

Внедрение SLA	Согласование стандартов качества между отделами.
Цикл Деминга (PDCA)	Постоянное улучшение: Plan-Do-Check-Act.

5. Примеры

- McDonald's: Единые стандарты приготовления блюд во всех ресторанах.
- Производство: Конвейерные линии с точными инструкциями для рабочих.

6. Проблемы и решения

- Проблема: Сопротивление сотрудников.
Решение: Обучение и вовлечение в процесс стандартизации.
- Проблема: Изменение внешних условий.
Решение: Гибкие регламенты с возможностью адаптации.

26 - Участники бизнес-процессов.

Кто такие участники бизнес-процессов?

Участники бизнес-процессов – это лица, роли или системы, которые вовлечены в выполнение бизнес-процесса. Они могут быть как активными (выполняют действия), так и пассивными (получают результаты или наблюдают за процессом). Участники могут быть представлены людьми, командами, отделами или автоматизированными системами.

Какие типы участников выделяются в бизнес-процессах?

1. Акторы (активные участники):

- Лица или системы, непосредственно выполняющие действия в процессе (например, сотрудник, который обрабатывает заказ).
- Пример: менеджер по продажам, который оформляет заказ клиента.

2. Заинтересованные стороны:

- Лица или группы, которые заинтересованы в результатах процесса, но не участвуют в его выполнении (например, клиент, руководство).
- Пример: клиент, ожидающий доставки товара.

3. Контролирующие органы:

- Участники, которые проверяют или контролируют выполнение процесса (например, отдел контроля качества).
- Пример: аудитор, проверяющий соответствие процесса стандартам.

4. Автоматизированные системы:

- Программные решения, которые выполняют часть процесса (например, CRM-система, автоматически отправляющая уведомления).
- Пример: система учета, автоматически обновляющая данные после завершения операции.

Как определяется роль участников в BPM (Business Process Modeling)?

В BPM роли участников определяются с помощью *дорожек (swimlanes)* на диаграммах процессов. Каждая дорожка соответствует определенной роли или участнику. Это позволяет визуализировать, кто за что отвечает в процессе. Например:

- Дорожка «Менеджер»: отвечает за согласование заказа.
- Дорожка «Логист»: отвечает за организацию доставки.

Какие ошибки часто возникают при определении участников?

1. Избыточность: Включение в процесс участников, которые не влияют на результат.
2. Неопределенность ролей: Нечеткое описание обязанностей участников.
3. Игнорирование заинтересованных сторон: Неучет лиц, которые могут повлиять на процесс или его результаты.
4. Неверное распределение задач: Назначение задач участникам, которые не обладают необходимыми компетенциями.

Почему важно правильно определять участников бизнес-процессов?

Правильное определение участников позволяет:

- Четко распределить ответственность за выполнение задач.
- Упростить контроль и координацию процесса.
- Сократить время выполнения операций за счет исключения дублирования функций.
- Повысить качество результатов процесса за счет вовлечения компетентных участников.

27 - Определение полноты и достаточности описания бизнес-процесса.

Что такое полнота описания бизнес-процесса?

Полнота описания бизнес-процесса – это степень, в которой описание охватывает все необходимые элементы, этапы и аспекты процесса. Полное описание должно включать:

- Все шаги процесса от начала до конца.
- Участников, их роли и ответственность.
- Ресурсы, используемые в процессе.
- Входящие и исходящие данные (документы, информация).
- Условия и правила, регулирующие выполнение процесса.

Какие критерии используются для оценки полноты описания?

1. **Полнота шагов:** Описание должно охватывать все этапы процесса без пропусков.
2. **Четкость ролей:** Все участники и их обязанности должны быть явно указаны.
3. **Ресурсы и данные:** Обязательно указание всех необходимых ресурсов (оборудование, программное обеспечение) и данных (входные и выходные документы).
4. **Условия и правила:** Описание должно включать все условия ветвления, исключения и правила выполнения процесса.
5. **Результаты:** Должны быть указаны ожидаемые результаты каждого этапа и процесса в целом.

Что такое достаточность описания бизнес-процесса?

Достаточность описания бизнес-процесса – это уровень детализации, при котором описание позволяет всем участникам процесса четко понимать свои задачи и выполнять их без дополнительных уточнений. Достаточное описание должно быть:

- Понятным для всех участников.
- Минимально избыточным (не содержать лишней информации).
- Пригодным для реализации процесса на практике.

Как определить, что описание бизнес-процесса является достаточным?

1. **Тестирование на практике:** Описание должно быть проверено на выполнение реальных задач.
2. **Отзывы участников:** Участники процесса должны подтвердить, что описание понятно и пригодно для работы.
3. **Проверка на избыточность:** Описание не должно содержать лишних деталей, которые не влияют на выполнение процесса.
4. **Оценка результатов:** Результаты выполнения процесса должны соответствовать ожидаемым.

Какие ошибки встречаются при описании бизнес-процессов?

1. **Недостаточная детализация:** Пропуск важных шагов или условий, что приводит к непониманию процесса.
2. **Избыточность:** Включение лишней информации, которая затрудняет восприятие.
3. **Нечеткость ролей:** Участники не понимают свои задачи и ответственность.
4. **Неучет исключений:** Отсутствие описания действий в нестандартных ситуациях.
5. **Непоследовательность:** Шаги процесса описаны в неправильном порядке или логически не связаны.

Какие инструменты используются для описания бизнес-процессов?

1. **BPMN (Business Process Model and Notation):** Стандарт для моделирования бизнес-процессов с использованием графических элементов.
2. **Use Case:** Описание сценариев использования процессов.
3. **Диаграммы потоков данных (DFD):** Визуализация потоков информации в процессе.
4. **ER-диаграммы:** Описание связей между сущностями в базе данных, используемыми в процессе.
5. **Текстовые описания:** Документирование процессов в виде текста с указанием шагов и участников.

Полное и достаточное описание бизнес-процесса является ключевым для его успешной реализации. Оно должно быть понятным, охватывать все необходимые элементы и исключать избыточность. Регулярная проверка и доработка описаний помогают избежать ошибок и повысить эффективность процессов.

28 - Позитивные и негативные исходы бизнес-процессов.

Понятие бизнес-процесса:

Исход — это конечное состояние экземпляра (инстанса) бизнес-процесса, при котором прекращается выполнение всех его шагов, а результат фиксируется в системе.

Любой регламентированный бизнес-процесс обладает свойством **вариативности**. Это значит, что в зависимости от входящих данных, внешних условий и решений участников, процесс может пойти по разным сценариям и прийти к разным конечным точкам.

Позитивные исходы:

Позитивный исход — это достижение процессом той цели, ради которой он создавался. Признаки позитивного исхода:

- Процесс завершается штатно, без сбоев, задержек и дополнительных издержек.
- Процесс проходит по основному пути.

Негативные исходы:

Негативные исходы бывают двух типов:

1. Нормативный отказ
Процесс отработал абсолютно правильно, без ошибок и нарушений регламента, но система или участник приняли решение отказать
2. Отказ в результате сбоя
Прерывание процесса из-за внутренней ошибки, нарушения регламента, нехватки ресурсов или технических проблем.

Ключевое отличие двух типов: в первом случае процесс доходит до конца, а во втором — ломается в процессе выполнения.

Примеры:

1. Автоматический отказ в выдаче кредита из-за плохой кредитной истории заемщика.
2. База данных зависла во время проверки клиента.

Как IT-аналитик может минимизировать негативные исходы?

1. **Глубокий анализ требований:** Понимание потребностей бизнеса и пользователей.
2. **Тестирование:** Проведение тестов на всех этапах разработки и внедрения системы.
3. **Поэтапная реализация:** Внедрение системы постепенно, чтобы минимизировать риски.
4. **Обучение пользователей:** Обучение сотрудников работе с новой системой.

5. **Мониторинг и поддержка:** Регулярный анализ работы системы и оперативное устранение проблем.
6. **Обратная связь:** Учет мнений пользователей для улучшения системы.

29 - Условия и ветвления в бизнес-процессах. Не бинарные ветвления.

Что такое ветвление?

Условия и ветвления – это механизмы, позволяющие управлять потоком бизнес-процесса в зависимости от выполнения определенных условий. Они используются для создания сложных сценариев, где процесс может развиваться по разным путям в зависимости от данных, событий или решений.

Условия и ветвления необходимы для:

1. **Гибкости процессов:** Процесс может адаптироваться к различным сценариям.
2. **Обработки исключений:** Возможность реагировать на нестандартные ситуации.
3. **Принятия решений:** Автоматизация выбора дальнейших шагов на основе данных.
4. **Эффективности:** Упрощение процессов за счет исключения ручного вмешательства.

Типы условий в бизнес-процессах бывают:

1. **Логические условия:** Проверка значений переменных или данных.
 - Пример: Если сумма заказа больше 10 000 рублей, применить скидку.
2. **Событийные условия:** Реакция на события (например, получение данных).
 - Пример: Если клиент отменил заказ, прервать процесс доставки.
3. **Временные условия:** Реакция на время (например, таймеры).
 - Пример: Если ответ не получен в течение 24 часов, отправить напоминание.

Типы ветвлений бывают:

1. **Простое ветвление (Exclusive Gateway):**
 - Процесс выбирает один из путей на основе условия.
 - Пример: Если клиент новичок, отправить приветственное письмо, иначе пропустить.
2. **Параллельное ветвление (Parallel Gateway):**

- Процесс разделяется на несколько независимых параллельных потоков.
- Пример: Одновременная проверка кредитной истории и подтверждение адреса клиента.

3. Условное ветвление (Inclusive Gateway):

- Процесс может разделяться на несколько путей, в зависимости от выполнения условий.
- Пример: Если клиент выбрал доставку на дом и оплату курьеру, выполнить оба действия.

4. Цикл (Looping):

- Повторение части процесса до выполнения условия.
- Пример: Повторная попытка отправки письма, пока не будет получено подтверждение.

Бинарное ветвление — **ровно два исхода**: Да / Нет, True / False.

Небинарное ветвление — это точка, где возможны **три и более** альтернативных пути, либо **несколько путей активируются одновременно**.

Почему они возникают?

- Условий больше двух (статусы, категории, диапазоны)
- Несколько действий должны выполняться параллельно
- Часть веток необязательна

Типы небинарных ветвлений

1. Эксклюзивное ветвление с множеством исходов (Выбирается **ровно одна** ветка из нескольких)

<Приоритет заявки?>

- | | |
|----------------|-------------------------------|
| └─ Критический | → [Обработать немедленно] |
| └─ Высокий | → [Обработать в течение часа] |
| └─ Средний | → [Обработать сегодня] |
| └─ Низкий | → [Обработать на этой неделе] |

2. Параллельное ветвление (Все ветки активируются одновременно)

[Заказ оплачен] → <AND-разветвление>

- |— [Передать на склад]
- |— [Уведомить клиента]
- |— [Сформировать накладную]

3. Инклюзивное ветвление (Активируется **одна или несколько** веток в зависимости от условий)

<Какие уведомления нужны?>

- |— [условие: есть email] → [Отправить email]
- |— [условие: есть телефон] → [Отправить SMS]
- |— [условие: VIP-клиент] → [Позвонить менеджеру]

30 - Виды BPM-нотаций. BPMN, IDEF0, блок-схемы.

BPM-нотация (Business Process Modeling Notation) — это стандартизированный язык графического описания бизнес-процессов. Нотации позволяют наглядно представить, как выполняются процессы в организации: какие шаги последовательно или параллельно выполняются, кто участвует, какие события могут произойти, где принимаются решения.

В целом выделяют (тут не все):

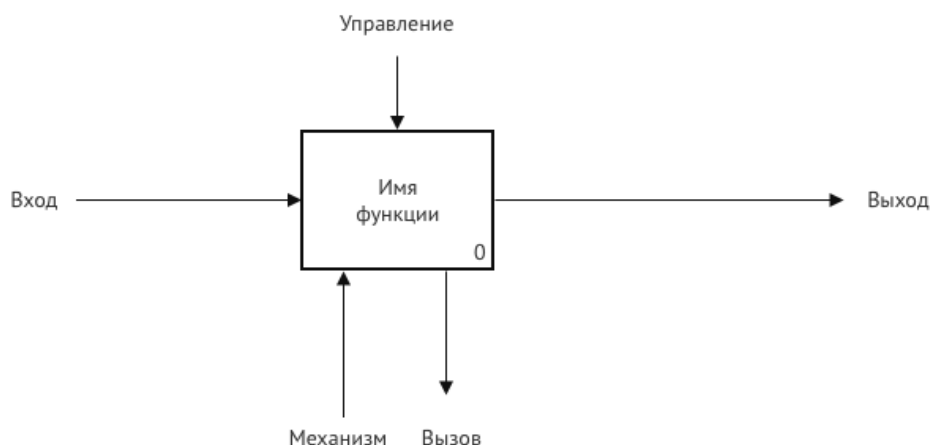
- **BPMN 2.0** (международный стандарт, по сути бесконкурентный)
- **IDEFO** (акцент на функциональности: цепочка «Событие → Функция → Событие»)
- **EPC (Event-Driven Process Chain)** (акцент на событиях и функциях)
- **UML (Unified Modeling Language)** (универсальный язык моделирования ПО (т.е. чаще используется разработчиками, а не для бизнес-процессов). Для бизнес-процессов используются диаграммы активности (Activity Diagram) и use case)
- **DFD (Data Flow Diagram)** (акцент на потоках данных, т.е. не показывает последовательность действий, только движение информации)

BPMN (Business Process Model and Notation)

Самая популярная современная нотация. Позволяет описывать процессы от высоких уровней до детальных исполнений (с точными условиями, таймерами, сообщениями), учитывает события (старт, конец, промежуточные), условия, параллельные потоки, роли исполнителей. Подходит для анализа, автоматизации и передачи процессов разработчикам.

Интересный факт: многие BPM-нотации (особенно BPMN 2.0) достаточно формализованы для автоматической трансляции в исполнимый код/XML (то есть программировать не надо!).

IDEFO (Integration Definition for Function Modeling)



Семейство нотаций IDEF, где IDEF0 — для функционального моделирования. Диаграмма состоит из блоков (функций) и стрелок, показывающих входы, выходы, управление и механизмы. Хорошо подходит для top-down декомпозиции процессов, но не показывает последовательность и события во времени.

И тут нет ветвлений (логических), только линейная иерархия (в том числе не может быть параллельных путей).

“Линейная иерархия в IDEF0 — это строгая последовательная декомпозиция от общего к частному, где каждая функция раскрывается на следующем уровне детализации по принципу «матрёшки»: один блок на вышестоящем уровне превращается в диаграмму с несколькими подфункциями на нижнем уровне, при этом каждый блок имеет только одного «родителя».”

Блок-схемы

Самая простая и универсальная нотация. Прямоугольники обозначают действия, ромбы — решения/условия, стрелки — направление потока. Используются для быстрых описаний алгоритмов и простых процессов, но не хватает средств для сложных сценариев с событиями и ролями.

31 - Синтаксис BPM 2.0. Дорожки, события.

Я так понимаю речь все-таки про BPMN 2.0 идет, ну да ладно

Дорожки — это элемент для изображения **исполнителей процесса**. Они показывают, **кто** выполняет каждую задачу.

Элемент	Описание
Пул (Pool)	Обозначает границы бизнес-процесса, организацию или участника процесса
Дорожка (Swimlane)	Внутри пула для распределения операций между участниками

Пример:

- Пул = организация (например, «Компания ООО «Ромашка»»)
- Дорожка = роль/отдел/должность внутри пула (например, «Менеджер», «Бухгалтерия», «Склад»)
- Дорожки позволяют иерархически делить пулы и другие дорожки
- Каждая задача помещается в дорожку исполнителя — сразу видно, кто делает

Событие — это то, что происходит в течение бизнес-процесса. Оказывает влияние на ход процесса, имеет причину (триггер) или результат. Изображается в виде круга.

События влияют на поток управления: они могут инициировать процесс, приостанавливать/изменять его ход или завершать его.

Классификация по местоположению:

Тип события	Обозначение	Когда используется	Потоки
Стартовое (Start)	Одинарная тонкая линия	Начало процесса	Только исходящий
Промежуточное (Intermediate)	Двойная тонкая линия	Середина процесса	Входящий + исходящий
Конечное (End)	Одинарная жирная линия	Конец процесса	Только входящий

ШОК, хорошо что мы ничего из этого не делали

Классификация по поведению:

Тип	Описание
Событие-обработчик	Приостанавливает процесс, ожидает наступления события (сообщение, сигнал, время, условие)
Событие-инициатор	Генерирует результат, не приостанавливая процесс (отправляет сообщение, сигнал, ошибку)

Триггеры событий инициализаторов закрашиваются.

Типы триггеров (внутри событий).

Иконка	Тип события	Описание
	Сообщение	Получение/отправка сообщения
	Таймер	Наступление времени/расписание
	Сигнал	Асинхронный сигнал
	Ошибка	Возникновение ошибки
	Компенсация	Компенсация действия
	Условия	Выполнение условия (temporary)

Микро-пример (процесс «Обработка заказа»):

стартовое событие — «Получено сообщение заказа» (Message Start); внутри между задачами может быть промежуточное событие «Таймаут оплаты» (Boundary Timer) на задаче «Ожидание оплаты», которое, если срабатывает, переводит поток в ветку «Отмена заказа»; конечное событие — «Заказ отправлен» (Message End), которое отправляет уведомление клиенту.

32 - Синтаксис BPM 2.0. Действия, шлюзы.

Действие — это общий термин для работы, которая выполняется в процессе. Графически изображается в виде прямоугольника с закруглёнными углами.

Типы действий:

Тип	Обозначение	Описание
Задача (Task)	Прямоугольник с закруглёнными углами	Базовая единица работы, не разбивается на подпроцессы
Подпроцесс (Subprocess)	Прямоугольник с закруглёнными углами + «+» внизу	Содержит вложенную диаграмму с детализацией процесса
Сложное действие (Call Activity)	Прямоугольник с двойной линией	Вызывает другой процесс/задачу из библиотеки (переиспользование)
Группа (Group)	Прямоугольник с пунктирной линией	Только для визуального объединения, не влияет на поток

Шлюз — это логический оператор для контроля ветвления и слияния потоков управления. Графически изображается в виде ромба. Шлюз может разделять поток на несколько маршрутов или объединять несколько потоков в один.

Типы шлюзов:

Тип	Маркер внутри ромба	Описание	Когда используется
Эксклюзивный (XOR)	X или пустой	Только один маршрут из альтернативных	Условие «либо-либо» (да/нет)
Неэксклюзивный (OR)	✓ или +	Один или несколько альтернативных маршрутов	Когда может быть несколько одновременных условий

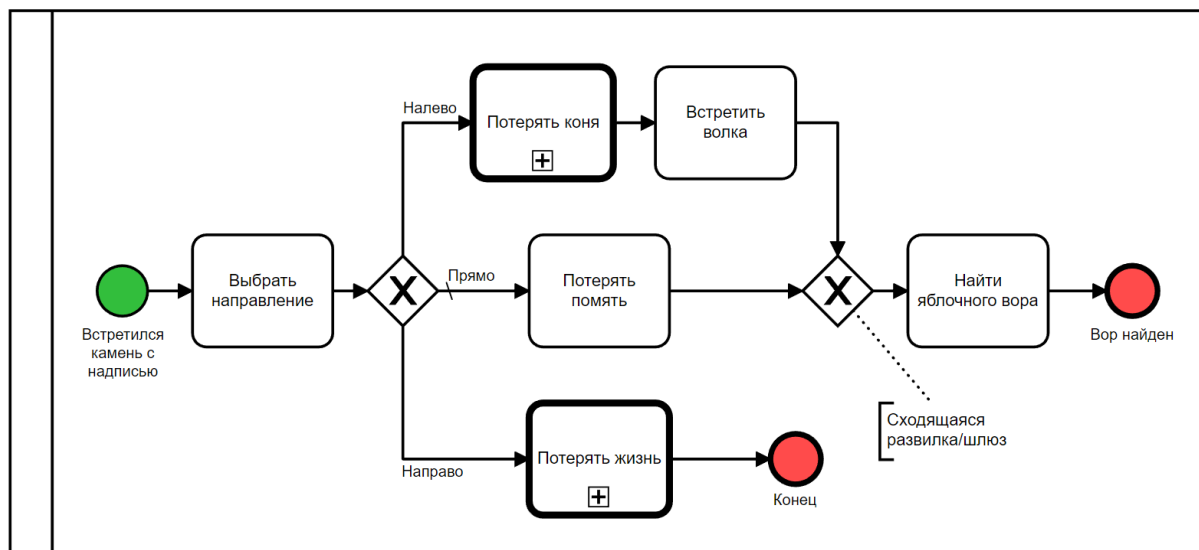
Параллельный (AND)	+	Все маршруты активируются одновременно	Распараллеливание процесса
Событийный (Event-based)	⚡ (событие)	Выбор маршрута по первому наступившему событию	Ждем, что произойдет раньше: сообщение, таймер, сигнал
Комплексный (Complex)	*	Сложные условия, не описываемые другими шлюзами	Нестандартная логика ветвления/слияния

Как по мне, [очень хорошая статья объясняющая шлюзы](#), если интересно обязательно посмотрите (там даже есть анимации).

Важно: Шлюз обязательно должен осуществлять ветвление или слияние.

Забавный пример со шлюзами XOR (штрихом обозначается поток по умолчанию):

тут потерять коня и потерять жизнь – подпроцессы.



33 - Синтаксис BPM 2.0. Таймеры в бизнес-процессах.

Таймер (Timer) — это событие в BPMN, которое показывает ожидание процессом:

- регулярного события (расписание),
- определенного момента времени (конкретная дата/время),
- временного периода (прошло N часов/дней).

На время ожидания текущий поток управления **приостанавливается**.

Круг с иконкой часов  внутри. Таймер всегда является событием-обработчиком (ожидает наступления времени, см. билет 31), так как время идёт независимо от процесса.

Это для граничных/промежуточных таймеров (снова см. 31 вопрос)

Тип	Для чего	Пример
Прерывающий	Максимально допустимое время на задачу	«Если пицца не выбрана за 30 минут → отмена заказа»
Непрерывающий	Параллельное действие по истечении времени	«Напомнить клиенту через 1 час, но задача продолжается»

Как задается время:

Тип таймера	Формат записи	Пример	Значение
Дата и время	YYYY-MM-DDThh:mm:ss	2026-05-28T09:00:00	28 мая 2026 в 09:00
Длительность	P[n]D / P[n]H / P[n]M	P2H P3D	2 часа 3 дня
Повторяющийся	R[количество] / [начало] / [длительность]	R5/2026-05-01T09:00:00/P1D	5 раз, ежедневно начиная с 1 мая

Важно: таймеры не задают сколько задача должна выполняться (для этого есть T3), они показывают, что делать если задача выполняется больше какого-то времени.

34 - Синхронизация бизнес-процессов.

Синхронизация бизнес-процессов — это согласование выполнения нескольких процессов или потоков работ во времени и по условиям выполнения.

Синхронизация необходима для:

- координации процессов (какие процессы обязаны выполняться последовательно, а какие можно выполнять параллельно);
- обмена данными между процессами;
- предотвращения конфликтов;
- обеспечения целостности данных;
- согласования действий участников.

Пример: организация мероприятия

Необходимо синхронизировать:

- бронирование помещения;
- подготовку оборудования;
- регистрацию участников.

Виды синхронизации

1. Последовательная синхронизация

Один процесс начинается только после завершения другого.

Пример: Подписание договора возможно только после его подготовки.

2. Параллельная синхронизация

Несколько процессов выполняются одновременно.

Пример: Одновременно выполняются проверка документов, расчёт стоимости, подготовка отчёта.

3. Синхронизация с объединением потоков

Несколько параллельных процессов должны завершиться перед продолжением работы.

Пример: Отгрузка товара выполняется после оплаты, упаковки, подготовки документов.

4. Событийная синхронизация

Процесс продолжается после наступления определённого события.

Пример: Начало доставки после подтверждения оплаты.

35 - Программная реализация бизнес-процессов.

Программная реализация бизнес-процессов — это создание программных средств, обеспечивающих автоматизацию, выполнение и контроль бизнес-процессов.

Иными словами, это перенос описанного процесса в информационную систему или программное обеспечение.

Программная реализация бизнес-процессов необходима для:

- автоматизации процессов;
- хранения и обработки данных;
- контроля выполнения процессов;
- организации взаимодействия участников.

Способы программной реализации бизнес-процессов

1. Полная автоматизация
Все действия выполняются системой автоматически.
2. Частичная автоматизация
Часть действий выполняет человек, часть — система.
3. Интеграция нескольких систем
Процесс распределяется между различными приложениями и сервисами.

Требования к программной реализации

- Надежность
Система должна корректно работать при различных условиях.
- Производительность
Процессы должны выполняться достаточно быстро.
- Масштабируемость
Система должна поддерживать рост количества пользователей и данных.
- Безопасность
Необходимо защищать данные и контролировать доступ.
- Гибкость
Должна существовать возможность изменения процессов.

36 - Реестр бизнес-процессов. Цели формирования реестра.

Реестр бизнес-процессов — это структурированный перечень всех бизнес-процессов организации с их описанием и основными характеристиками.

Цели формирования реестра

1. Формализация деятельности организации
2. Систематизация бизнес-процессов
3. Анализ и подготовка к оптимизации и автоматизации процессов
4. Распределение ответственности
5. Прозрачность (все видят, какие процессы есть в компании)

Виды бизнес-процессов в реестре

1. Основные процессы
2. Вспомогательные процессы
3. Управленческие процессы

Этапы формирования реестра

1. Анализ деятельности организации
2. Выявление процессов
3. Классификация процессов
4. Описание процессов
5. Формирование структуры реестра

Реестр процессов имеет 2 представления: дерево и каталог. (хз это в общем или это частный случай одного инструмента для создания реестров)

ну я совсем хз что сюда писать, проверьте цели пж

37 - Запуск бизнес-процессов от пользователя. Реализация шагов бизнес-процессов.

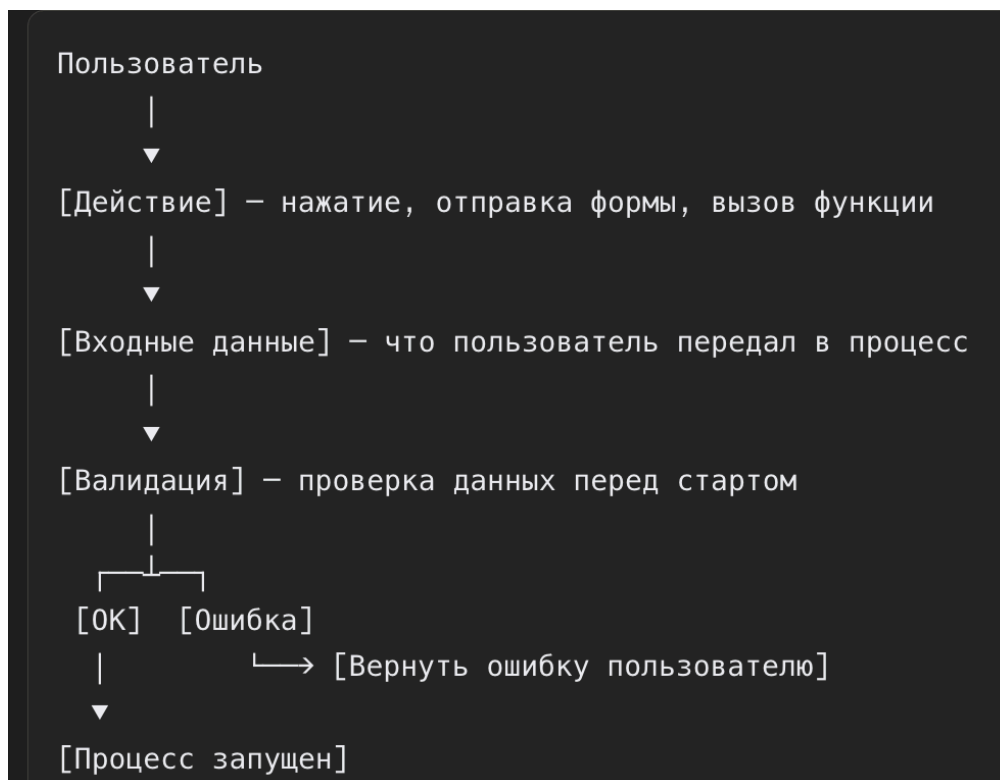
Запуск бизнес-процессов

Запуск (инициация) бизнес-процесса — это событие, которое переводит процесс из состояния «не активен» в состояние «выполняется».

Существует три базовых типа запуска:

Тип	Кто/что инициирует	Пример
От пользователя	Человек совершает действие	Нажал кнопку «Оформить заказ»
По событию	Внешнее или системное событие	Пришло письмо, сработал таймер
По расписанию	Время / триггер планировщика	Отчёт формируется каждый понедельник

Из чего состоит запуск от пользователя:



Реализация шагов бизнес-процесса

Шаг — это атомарная единица работы внутри процесса, которая:

- имеет чёткий вход и выход
- выполняется одним исполнителем (роль или система)
- имеет понятный признак завершения

Шаги бывают человекоориентированные и автоматизированные (в зависимости от необходимости участия человека)

Каждый шаг должен быть описан по следующей структуре:

Название шага	
Исполнитель	Роль / система
Вход	Что поступает на шаг
Действие	Что конкретно делается
Выход	Результат / артефакт
Условие	При каких условиях выполнять
Исключения	Что делать при ошибке
Время (SLA)	Максимальное время выполнения

38 - Проектирование программной реализации бизнес-процессов. Графическое представление.

Программная реализация бизнес-процесса — это перевод описания процесса (текстового или графического) в работающую систему, где шаги, условия, роли и данные воплощены в коде, БД и интерфейсах.

Проектирование — это промежуточный этап между описанием процесса и его программной реализацией.

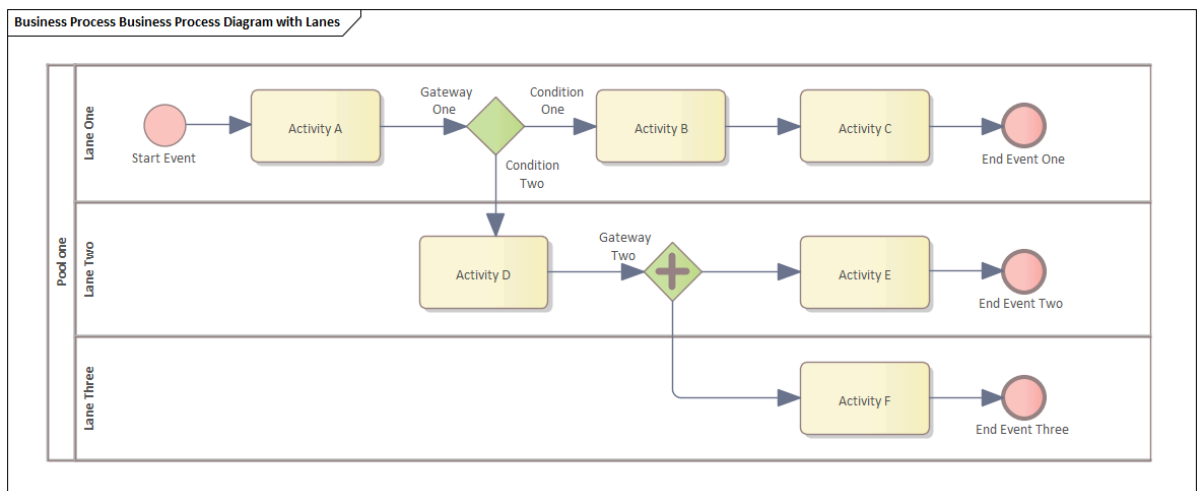
- Описание БД — что делать
- Проектирование — как делать
- программная реализация — рабочий экземпляр

Этапы проектирования программной реализации

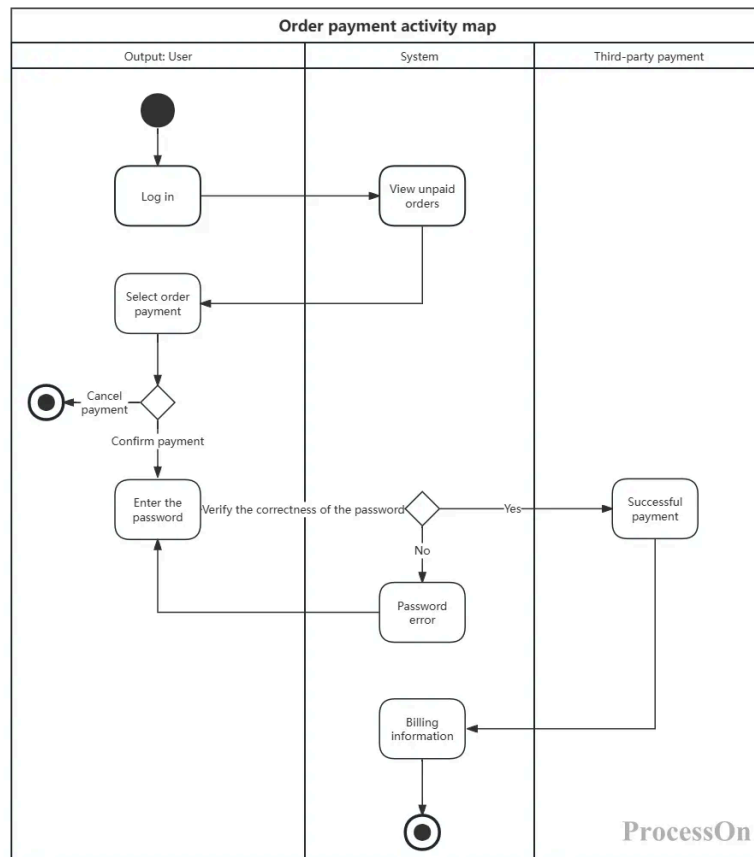
1. Хранение состояния процесса
Каждый запущенный процесс — это **экземпляр** со своим состоянием. Нужно решить, где и как его хранить.
2. Хранение данных процесса
Данные, передаваемые между шагами — контекст процесса. Обычно хранятся в формате JSON
3. Маршрутизация шагов
Как система определяет, какой шаг следующий:
 - Жёсткая последовательность — следующий шаг прописан в коде
 - Таблица переходов — маршрут хранится в БД и настраивается
 - BPM-движок — специализированная система (Camunda, Activiti, jBPM)

Графическое представление

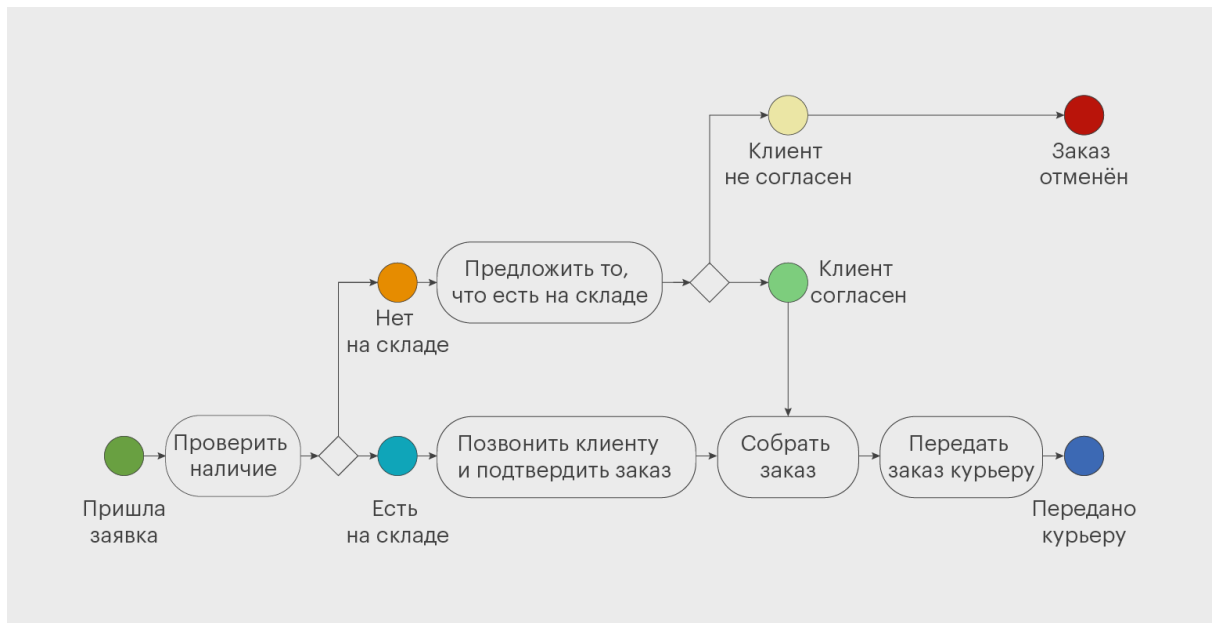
1. BPMN (Business Process Model and Notation)



2. UML Activity Diagram. Используется для детализации логики на техническом уровне



3. Граф состояний



39 - Уведомления в бизнес-процессах. Правила формирования уведомлений.

Уведомление — это **сигнал** участнику процесса о том, что произошло событие, требующее его внимания или информирования.

Без уведомлений процесс ломается: исполнители не знают, что задача назначена; инициатор не знает, что процесс завершён; руководитель не знает, что произошла ошибка.

Классификация уведомлений

1. По цели (Оперативное, Информационное, Системное)
2. По способу доставки (Push, Email, SMS, Мессенджер, звонок)
3. По триггеру (новый шаг, смена статуса, дедлайн, ошибка)

Правила формирования уведомлений

1. Адресность
Уведомление должно быть отправлено **нужному получателю** — не всем подряд.
2. Своевременность
Уведомление должно приходить **в нужный момент** — не слишком рано и не с опозданием.
3. Информативность
Уведомление должно отвечать на вопросы: **что случилось, что нужно сделать, где это сделать.**
4. Ненавязчивость
Слишком много уведомлений = ни одно не читается.
5. Идемпотентность
Одно событие → **одно уведомление**. Дублирование недопустимо.
6. Контекстность
Уведомление должно содержать **достаточно данных из контекста процесса**, чтобы получатель мог действовать без дополнительных поисков.
7. Трассируемость
Каждое отправленное уведомление должно быть **зафиксировано**.

Схема проектирования уведомлений

1. Определить события-триггеры (когда отправлять?)
↓
2. Определить получателей (кому отправлять?)
↓
3. Выбрать канал (как отправлять?)
↓
4. Составить шаблон (что написать?)
↓
5. Задать правила группировки / дайджеста
↓
6. Спроектировать эскалацию (что если игнорируют?)
↓
7. Предусмотреть логирование (как отследить?)

40 - Статусы состояний бизнес-процессов. Цели и задачи формирования статусов.

Статусы состояний бизнес-процессов – это фиксированные этапы, через которые проходит процесс; используются чтобы отражать его текущее состояние и управлять переходами между шагами. Они особенно полезны там, где процесс не является линейным и может переходить между несколькими состояниями в зависимости от событий, условий или действий участников.

Статус показывает, на каком этапе находится объект или процесс: например, «новый», «на согласовании», «на доработке», «завершён». В процессах со статусами обычно есть набор состояний, переходов и действий, а одно из состояний задается как начальное.

Основная цель – сделать ход процесса понятным, управляемым и контролируемым для всех участников. Статусы позволяют видеть текущее положение задачи и упрощать координацию между сотрудниками. Еще одна важная цель – обеспечить гибкую логику процесса и разграничение прав доступа на разных этапах.

Ключевые задачи формирования статусов такие:

- Отражать текущее состояние процесса или объекта.
- Задавать допустимые переходы между этапами.
- Определять действия, которые можно выполнять в каждом состоянии.
- Разделять права доступа в зависимости от статуса.
- Упрощать контроль сроков, анализ узких мест и подготовку отчетности.

На практике статусы помогают руководителю быстро понять, где «застрял» процесс, а исполнителю — что нужно делать дальше.

Если что это про те статусы, которые мы рядом с формами писали.

Со слов Роров: эти статусы нужны, чтобы если возникла проблема (процесс где-то застрял) менеджер мог бы зайти и сразу увидеть на каком этапе он застрял и дальше уже работать с этим. Это как бы главная информация и краткое описание текущего состояния.

Поэтому статусы пишутся на (у нас) русском, потому что нужны они не программистам, а менеджерам.

Пример: заявка на отпуск может иметь статусы: «Создана» → «На согласовании» → «Одобрена» → «Завершена».

41 - Проектирование экранных форм шагов бизнес-процесса.

Создание эскизов форм.

Проектирование экранных форм шагов бизнес-процесса — это разработка интерфейсов, через которые пользователь выполняет действия на каждом этапе бизнес-процесса. Такие формы определяют состав полей, порядок их расположения, доступные кнопки, правила ввода данных и условия отображения элементов.

Как строятся эскизы форм (план):

В принципе мы почти все тут делали в работе, должны помнить

1. Описать бизнес-процесс.
Зафиксировать цель процесса, его начало и конец, участников, входные и выходные данные, а также основные шаги выполнения.
2. Разбить процесс на этапы и статусы.
Для каждого шага определить, какой статус он отражает, какие переходы возможны и где есть развилки, согласование или возврат на доработку.
3. Построить граф бизнес-процесса.
Изобразить последовательность шагов в виде схемы: действие, решение, переход, завершение.
4. Определить экранные формы для каждого шага.
Для каждого этапа решить, нужна ли отдельная форма.
5. Собрать состав полей.
Выписать все данные, которые пользователь должен ввести, выбрать или только просмотреть на конкретном шаге. (+какие поля обязательные, а какие необязательные)
6. Задать ограничения.
Для каждого поля и действия указать ограничения: формат ввода, диапазон значений, доступность кнопок, права пользователя, условия отображения полей, автоматическое заполнение и проверки.
7. Сделать эскиз формы.
Нарисовать упрощённый макет экрана: заголовок, поля, кнопки, подсказки, сообщения об ошибках, области просмотра и блоки данных.
На этом этапе важна не красота, а логика и удобство.

При проектировании экранных форм важно учитывать правила оформления интерфейса. Нужно делать понятные подписи полей и кнопок, располагать элементы в логичном порядке, группировать связанные данные, добавлять подсказки к сложным полям и предусматривать сообщения об ошибках. Часть полей может быть доступна только для просмотра, скрыта или недоступна для редактирования в зависимости от роли пользователя и статуса процесса.

42 - Описание данных бизнес-процесса. Формирование общего описания данных.

Зачем описывать данные процесса?

Бизнес-процесс — это не только последовательность шагов, но и данные, которые в нём живут: создаются, передаются, изменяются и хранятся. Без описания данных невозможно ни спроектировать систему, ни передать задачу разработчику.

Цели описания данных:

- Понять, что именно обрабатывает процесс
- Определить, какие данные нужны на каждом шаге
- Спроектировать хранилище и интерфейсы
- Обеспечить согласованность данных между участниками

Типы данных в бизнес-процессе

1. По роли в процессе (входные/выходные данные, управляющие данные, контекстные данные, рабочие/промежуточные данные)
2. По способу хранения (переменные, сущности в БД, файлы/вложения, сообщения)

Уровни описания данных

- Концептуальный: что за данные (сущности и связи)
- Логический: как организованы (атрибуты, типы, правила)
- Физический: где хранятся (таблицы, поля, форматы)

Формирование общего описания данных

Общее описание данных — это сводный документ, который объединяет все данные процесса в единую структуру. Формируется по следующему алгоритму:

1. Инвентаризация данных
Пройтись по каждому шагу процесса и выписать все упоминания данных
2. Группировка в сущности
Сгруппировать выявленные поля по смысловым объектам — сущностям.
3. Описание каждой сущности

4. Описание связей

Фиксируются отношения между сущностями: один-ко-многим, многие-ко-многим, обязательность связи.

5. Описание бизнес-правил на данные

6. Словарь данных

Итоговый артефакт — единый реестр всех сущностей и атрибутов

43 - Реализация описания данных. Пример.

Я так понял тут роров хочет видеть пример к тому что выше так что НЯ

Процесс: “Заявка на закупку”

Шаги:

1. От сущностей к схеме БД

Каждая сущность из словаря данных превращается в таблицу, атрибуты — в поля, связи — в внешние ключи.

```
sql
-- Пользователь
CREATE TABLE users (
  id          UUID PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  role        VARCHAR(50)  NOT NULL -- 'employee', 'manager', 'director'
);

-- Заявка
CREATE TABLE purchase_requests (
  id          UUID PRIMARY KEY,
  number       VARCHAR(20)   NOT NULL UNIQUE, -- ZAK-2024-0001
  status       VARCHAR(30)   NOT NULL,        -- см. статусную модель
  initiator_id UUID          NOT NULL REFERENCES users(id),
  approved_by_id UUID        REFERENCES users(id),
  total_amount NUMERIC(12, 2) NOT NULL CHECK (total_amount > 0),
  comment      TEXT,
  created_at   TIMESTAMPTZ   NOT NULL DEFAULT now(),
  approved_at  TIMESTAMPTZ
);

-- Позиция заявки
CREATE TABLE request_items (
  id          UUID PRIMARY KEY,
  request_id   UUID          NOT NULL REFERENCES purchase_requests(id),
  product_name VARCHAR(255)  NOT NULL,
  quantity     INTEGER        NOT NULL CHECK (quantity > 0),
  unit_price   NUMERIC(10, 2) NOT NULL CHECK (unit_price > 0),
  total_price  NUMERIC(12, 2) GENERATED ALWAYS AS (quantity * unit_price) STORED
);
```

2. Реализация статусной модели

Статусы реализуются через перечисление (ENUM) или таблицу переходов.

a. ENUM

```
sql

CREATE TYPE request_status AS ENUM (
    'DRAFT',          -- Черновик
    'PENDING',         -- На рассмотрении
    'APPROVED',        -- Одобрена
    'REJECTED',        -- Отклонена
    'EXECUTED',        -- Исполнена
    'WITHDRAWN'       -- Отозвана
);

-- Используется в таблице:
status request_status NOT NULL DEFAULT 'DRAFT'
```

b. Таблица переходов

```
sql

CREATE TABLE status_transitions (
    from_status VARCHAR(30) NOT NULL,
    to_status   VARCHAR(30) NOT NULL,
    role_allowed VARCHAR(50) NOT NULL, -- кто может выполнить переход
    PRIMARY KEY (from_status, to_status)
);

INSERT INTO status_transitions VALUES
    ('DRAFT', 'PENDING', 'employee'),
    ('DRAFT', 'WITHDRAWN', 'employee'),
    ('PENDING', 'APPROVED', 'manager'),
    ('PENDING', 'REJECTED', 'manager'),
    ('APPROVED', 'EXECUTED', 'manager');
```

3. Модели в коде (бэк)

Описание данных транслируется в **классы/модели** на языке программирования:

```
from enum import Enum
from dataclasses import dataclass
from datetime import datetime
from uuid import UUID
from typing import Optional

class RequestStatus(Enum):
    DRAFT = "DRAFT"
    PENDING = "PENDING"
    APPROVED = "APPROVED"
    REJECTED = "REJECTED"
    EXECUTED = "EXECUTED"
    WITHDRAWN = "WITHDRAWN"
```

```
@dataclass
class PurchaseRequest:
    id: UUID
    number: str
    status: RequestStatus
    initiator_id: UUID
    total_amount: float
    created_at: datetime
    comment: Optional[str] = None
    approved_by_id: Optional[UUID] = None
    approved_at: Optional[datetime] = None

    # Бизнес-правило в коде
    def can_approve(self, user_role: str) -> bool:
        return (
            self.status == RequestStatus.PENDING
            and user_role == "manager"
        )
```

```
def approve(self, approver_id: UUID):
    if not self.can_approve("manager"):
        raise ValueError("Переход недопустим")
    self.status = RequestStatus.APPROVED
    self.approved_by_id = approver_id
    self.approved_at = datetime.now()
```

Ключевой момент: бизнес-правила из словаря данных живут в модели, а не размазаны по коду интерфейса.

4. Контракт API (передача данных). JSOOOOONCHIIK

Когда данные передаются между системами или фронтендом и бэкендом, описание данных реализуется как **схема запроса/ответа**:

```
// POST /api/purchase-requests - создание заявки
// Запрос (то, что присылает пользователь):
{
  "comment": "Необходимо для офиса",
  "items": [
    {
      "product_name": "Ноутбук Dell XPS",
      "quantity": 2,
      "unit_price": 75000.00
    }
  ]
}

// Ответ (то, что возвращает система):
{
  "id": "uuid-...",
  "number": "ZAK-2024-0001",
  "status": "DRAFT",
  "total_amount": 150000.00,
  "created_at": "2024-01-15T10:30:00Z",
  "initiator": {
    "id": "uuid-...",
    "name": "Иванов И.И."
  }
}
```

5. Форма пользовательского интерфейса

Атрибуты сущности напрямую определяют состав полей формы:

Атрибут	Тип	Обязательный	→	Элемент UI
comment	Text	Нет	→	textarea (необязательный)
items	Array	Да	→	динамический список строк
product_name	String	Да	→	input[type=text], required
quantity	Integer	Да	→	input[type=number], min=1
unit_price	Decimal	Да	→	input[type=number], min=0.01
total_amount	Computed	—	→	поле только для чтения

44 - Формат JSON. JSON-SCHEMA.

Формат JSON (JavaScript Object Notation) — легковесный текстовый формат обмена данными, основанный на подмножестве синтаксиса JavaScript. Является язык-независимым стандартом (RFC 8259, ECMA-404).

Преимущества:

- человекочитаемый, легко редактируемый;
- поддерживается практически всеми языками и платформами;
- идеален для REST API, конфигураций, логов, кэша;
- легко сериализуется и десериализуется.

Ограничения:

- нет встроенной поддержки комментариев;
- нет нативного типа date/time (хранится как строка/число);
- нет встроенной валидации структуры: при получении данных нельзя гарантировать их корректность без дополнительных средств.

JSON Schema

JSON Schema — формат описания и валидации структуры JSON-документов. Сама схема также записывается в формате JSON. Регулируется спецификацией IETF JSON Schema.

Назначение:

- гарантировать, что входящие/исходящие данные соответствуют ожидаемому контракту;
- автоматически документировать структуру данных;
- генерировать код, формы, тесты.

Ключевые слова схемы:

Ключевое слово	Назначение
\$schema	Версия спецификации
type	Ожидаемый тип данных
properties	Описание полей объекта
required	Обязательные поля
items	Схема элементов массива
minimum/maximum	Границы числовых значений
minLength/maxLength	Границы строк

pattern	Регулярное выражение
enum	Фиксированный набор значений
format	Семантический формат
additionalProperties	Разрешены ли дополнительные поля
\$ref	Ссылка на другую схему

Пример JSON-документа:

```
{
  "id": 1,
  "name": "Иван Петров",
  "age": 28,
  "isActive": true,
  "roles": ["admin", "user"],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
```

Пример JSON Schema для документа выше:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "User",
  "type": "object",
  "properties": {
    "id": { "type": "integer", "minimum": 1 },
    "name": { "type": "string", "minLength": 2 },
    "age": { "type": "integer", "minimum": 0, "maximum": 150 },
    "isActive": { "type": "boolean" },
    "roles": {
      "type": "array",
      "items": { "type": "string", "enum": ["admin", "user",
"moderator"] },
      "minItems": 1
    },
    "address": {
      "type": "object",
      "properties": {
        "city": { "type": "string" },
        "zip": { "type": "string", "pattern": "^\\d{6}$" }
      },
      "required": ["city"]
    }
  },
  "required": ["id", "name", "isActive"],
  "additionalProperties": false
}
```

Процесс валидации

1. Приложение загружает схему и входящий JSON.
2. Валидатор (Ajv в JS, jsonschema в Python, everit-org в Java) проверяет соответствие.
3. Возвращает true или массив ошибок с указанием пути к проблемному полю.
4. При ошибке возвращает 404 Bad Request с детальным описанием.

Роль в технологии проектирования приложений

1. API-контракты:

JSON Schema описывает структуру запросов/ответов REST/gRPC API. Является основой OpenAPI 3.x

2. Валидация на входе:

Защита от malformed-данных, SQL/NoSQL-инъекций, DoS через глубокую вложенность.

3. Документация:

Автогенерация интерактивных документов и клиентских SDK.

4. Микросервисная архитектура:

Контрактное тестирование, гарантии совместимости версий данных.

5. Фронтенд:

Автогенерация форм (React JSON Schema Form), валидация в реальном времени, типизация (TypeScript генерация из схемы).

6. CI/CD & DevOps:

Валидация конфигурационных файлов (docker-compose, k8s манифесты часто используют схемы).

Преимущества JSON Schema:

- язык-независимый стандарт валидации;
- декларативность: схема = документация = валидатор = генерация кода;
- поддержка наследования (\$ref, \$defs), условной логики (if/then/else);
- снижает количество ручных проверок и багов на стыке сервисов.

Ограничения:

- не все валидаторы поддерживают последние спецификации одинаково;
- накладные расходы на проверку в высоконагруженных системах;
- не заменяет бизнес-логику валидации (например, проверку уникальности email в БД).

45 - Формат JSON. Синтаксис.

Формат JSON (JavaScript Object Notation) — легковесный текстовый формат обмена данными, основанный на подмножестве синтаксиса JavaScript. Является язык-независимым стандартом (RFC 8259, ECMA-404).

Правило	Описание
Кавычки " "	Все строки и ключи обязательно заключаются в двойные кавычки
Запятые	Разделяют элементы в объектах и массивах. Запятая после последнего элемента запрещена
Фигурные/квадратные скобки	{ } – объект, [] – массив
Пробелы	Пробелы, табуляции и переносы строк вне строк игнорируются парсером
Регистр	Ключевые литералы пишутся строго строчными: true, false, null
Кодировка	По умолчанию UTF-8

Структуры данных

1. Объект (неупорядоченная коллекция пар **ключ : значение**):

```
{
  "userId": 1042,
  "login": "dev_admin",
  "settings": {
    "theme": "dark",
    "notifications": true
  }
}
```

2. Массив (упорядоченная коллекция значений):

```
["error", "warning", "info"]
```

Типы данных в JSON:

Тип	Синтаксис	Примечание
string	"text"	Поддержка экранирования
number	целое/дробное	Без ведущих нулей, без NaN/Infinity

boolean	true / false	Только строчные буквы
null	null	Аналог None/nil/NULL
object	{...}	Ключи всегда строки
array	[...]	Элементы любого типа JSON

Правила экранирования строк

Внутри строк допустимы следующие escape-последовательности:

\" → двойная кавычка
 \\ → обратный слэш
 \/ → прямой слэш (опционально)
 \b → backspace
 \f → form feed
 \n → новая строка
 \r → возврат каретки
 \t → табуляция
 \uXXXX → Unicode-символ (например, \u00A9 → ©)

Пример:

```
{
  "path": "C:\\logs\\app.log",
  "message": "Line1\\nLine2",
  "emoji": "\\uD83D\\uDE80"
}
```

46 - Формат JSON. Массивы данных.

JSON-массив — упорядоченная коллекция значений, заключенная в квадратные скобки. Тип предназначен для передачи *списков, очередей, таблиц, результатов выборки и пакетных операций*. Порядок элементов сохраняется.

Синтаксические правила

Правило	Описание
Ограничители	Начинается и заканчивается квадратной скобкой
Разделитель	Запятая , между элементами
Trailing comma	Запрещена запятая после последнего элемента ["a", "b",]
Порядок	Строго сохраняется: индекс 0 – первый элемент
Пробелы	Допустимы вокруг запятых и элементов, игнорируются парсером
Пустой массив	Валиден []

В массиве может находиться любой валидный тип JSON:

- однородный: `["admin", "user", "moderator"]`
- числовой: `[10, 25.5, -3, 1e4]`
- смешанный: `[42, "active", true, null]`
- объектов: `[{"id":1, "name":"A"}, {"id":2, "name":"B"}]`
- вложенные структуры: `[[1,2], [3,4], [5,6]]`

Массивы в проектировании приложений

1. API-контракты:
Стандартный формат для коллекций в REST: `GET /api/users → [ {user}, {user}, ... ]`
2. Пагинация:
Массивы + метаданные: `{"items": [...], "page": 2, "total": 150}`
3. Batch-операции:
Пакетная отправка/обработка: `POST /api/orders/batch → [{"id":1}, {"id":2}]`
4. Маппинг в ЯП:
Автоматически преобразуются в: `List<T>` (Java), `list/tuple` (Python), `Array/Vec` (JS/Go).
5. Валидация:

В JSON Schema описываются через ключ items: `"type": "array",
"items": {"type": "string"}`

6. Конфигурации:

Списки зависимостей, маршрутов, плагинов: `"plugins": ["cors",
"auth", "logging"]`

Итог нейронки:

JSON-массивы представляют собой строгий, детерминированный механизм передачи упорядоченных данных. Их синтаксис гарантирует однозначный порядок элементов и совместимость со всеми современными ЯП, что делает их стандартом де-факто для коллекций в клиент-серверных взаимодействиях. В технологии проектирования приложений массивы используются как основа API-ответов, пакетных операций и конфигураций, а их предсказуемая структура позволяет автоматизировать валидацию, маппинг типов и генерацию клиентского кода.